

Cross-Domain State Preservation

Jinwook Kim (Jay)

May 12, 2026

Abstract

We present a generic, reusable framework for verifying that state transitions are faithfully preserved when synchronized across heterogeneous domains (blockchains, off-chain ledgers, etc.), and that the synchronization process remains live in the presence of Byzantine faults.

The framework combines two complementary properties. For *safety*, the core abstraction is a hierarchy of Isabelle/HOL locales: **state_machine** defines finite-state deterministic transition systems with terminal states; **state_preservation** captures the naturality (homomorphism) condition ensuring that a synchronization map commutes with transitions; **symmetric_state_preservation** extends this to bidirectional synchronization with inverse roundtrip guarantees; and **multi_domain_preservation** generalizes to N domains sharing the same abstract state machine, with a cross-domain consistency theorem. For *liveness* under Byzantine faults, three additional locales are introduced: **priority_system** guarantees deterministic resolution of conflicting requests via a total order on priority keys; **deadlock_free_locking** guarantees that timeout-based forced release prevents indefinite locking; and **fair_leader_system** guarantees starvation freedom under fair leader scheduling, formulated as an assume-guarantee abstraction of probabilistic VRF-based leader election.

As a practical application, we instantiate every locale of this framework with concrete examples drawn from the regulatory state model of Oraclizer, a state synchronization oracle for tokenized assets across blockchain networks and off-chain ledgers. The model comprises five states (ACTIVE, FROZEN, SEIZED, CONFISCATED, RESTRICTED) and seven actions. The instantiations include: (i) the regulatory transition system as a **state_machine**; (ii) a heterogeneous-action instance of **state_preservation** between two chains whose on-chain action vocabularies differ in scope (*escalation preservation*); (iii) a layer-crossing instance of **symmetric_state_preservation** between an on-chain enum representation and an off-chain DAML structured permission record (*onchain DAML bridge*); (iv) a parametric instance of **multi_domain_preservation** for any finite set of chain identifiers and any global state satisfying the validity invariant; (v) an instance of **priority_system** on the **priority_key** type of D-quencer messages, using a four-component lexicographic priority key derived from message metadata; (vi) a **deadlock_free_locking** instance with the system's lock timeout; and (vii) a **fair_leader_system**

instance with the D-quencer’s leader schedule. The synchronization protocol with preemptive locking is shown to preserve cross-chain regulatory consistency (the *regulatory homomorphism* theorem) and the global validity invariant (*valid state preservation*). The liveness side culminates in a *combined safety liveness* theorem that ties the synchronization function to the liveness guarantees and discharges the honest-node assumption of the safety proof under Byzantine faults.

Our use of locales for compositional, reusable abstractions follows the methodological tradition exemplified in the AFP by Lochbihler’s `ADS_Functor` entry, which structured authenticated data structures through composable building blocks. The technical content of this entry is independent of `ADS_Functor`; the connection is one of design philosophy (locale-based reusable abstractions over functor-shaped data), not formal dependency.

Scope of abstraction. The synchronization model in this entry abstracts each *sync* operation as an atomic action: lock acquisition, state update across connected chains, and lock release happen as a single step in the formal model. We do not model interleaved concurrent sync attempts, network message delays, or partial failures within a sync. This is a deliberate abstraction choice consistent with locale-based reasoning in the AFP for cross-domain protocols: under the modelled atomicity, the locales characterise the structural correctness of synchronization (naturality, roundtrip, multi-domain consensus); the operational realization of atomicity (timeout-based locks, automatic release on expiry, retry on contention) is an implementation concern out of scope for this entry. The liveness side of the framework (`deadlock_free_locking` in particular) provides the abstractions needed to lift the model toward a partially synchronous deployment, which we leave to subsequent entries.

A note on the naming of this revision and its relationship to the intended methodological positioning appears in the introduction.

Contents

1	Overview	2
1.1	Theory Dependency Structure	4
1.2	Design Decisions	4
1.3	Heterogeneous-Action Instance: Escalation Preservation . . .	5
1.4	Layer-Crossing Instance: Onchain DAML Bridge	5
1.5	Priority Instance: Well-Formed Messages	6
1.6	Combined Safety and Liveness	7
1.7	Beyond the Atomic Sync Model	7
2	Generic State Machine Locale	8
3	Cross-Domain State Preservation Locale	8

4	Symmetric State Preservation	11
5	Multi-Domain State Preservation	12
6	Regulatory State and Action Definitions	14
7	Fundamental Properties of the Transition Function	15
7.1	Invariant I1: CONFISCATED is terminal	15
7.2	Invariant I2: CONFISCATE reaches CONFISCATED from any non-terminal state	15
7.3	Invariant I3: Determinism (inherent from function definition)	15
7.4	SEIZED to FROZEN exclusion	15
7.5	FROZEN to RESTRICTED exclusion	16
7.6	Transition completeness: every non-terminal state has at least one valid action	16
7.7	Reachability from ACTIVE	17
7.8	Return paths to ACTIVE	17
8	State Machine Instance	17
9	Asset and Global State Modeling	18
10	State Accessors and Predicates	19
11	Global State Validity	19
12	Synchronization Protocol	20
12.1	Lock acquisition	20
12.2	State update across chains	20
12.3	The synchronization function	21
13	Synchronization Properties	21
13.1	Idempotency	21
13.2	Sync isolation: other assets are not affected	22
13.3	Sync correctness: source chain updated	22
13.4	Properties of update_all_chains	22
13.5	Lock mutual exclusion	23
14	Main Theorem: Cross-Chain Regulatory Consistency	24
15	Valid State Preservation	26
15.1	Part 1: consistent_state preservation	26
15.2	Part 2: no_locked_without_reason preservation	29
15.3	The valid_state preservation theorem	30
16	Heterogeneous-Action State Preservation Instance	31

17 Layer-Crossing Symmetric State Preservation Instance	36
18 Multi-Domain Locale Instantiation	43
19 Priority-Based Deterministic Selection	46
20 Deadlock-Free Locking with Timeout	48
21 Fair Leader Starvation Freedom	49
22 Authority Level and Action Severity	53
23 Priority Key	54
24 Extended Message Type	54
25 Priority Construction	54
26 Node and System Model	55
27 D-quencer System Locale	55
28 Priority System Instantiation	56
29 BFT Consensus Abstraction	59
30 Deadlock-Free Locking Instantiation	60
31 Fair Leader Starvation Freedom Instantiation	61
32 Combined Safety and Liveness	62
33 Acknowledgments	64

1 Overview

Naming and methodological positioning. An earlier version of this entry used the title “Cross-Domain State Preservation Functor” to reflect the entry’s methodological lineage from Lochbihler’s Merkle Functor (AFP entry `ADS_Functor`) and the intended design philosophy of locale-based reusable abstractions over functor-shaped state machines. The present revision establishes the foundational locales and concrete instances supporting that philosophy but does not yet formalize category objects, morphisms, or the functor laws (identity preservation, composition preservation) as theorems within the entry. A subsequent revision of this entry is planned to extend it with compositional integration and explicit functor laws, at which point

the original title will be restored. The title of the present revision therefore reflects what is formally proved within the entry, not the broader design pattern that motivates it.

This AFP entry consists of four theory files organized as two “generic locales + regulatory instances” pairs, one for safety and one for liveness.

State_Preservation.thy defines domain-independent locales for state machines, pairwise state preservation (with naturality), symmetric bidirectional preservation, and multi-domain preservation. All locales are parameterized over abstract types and can be instantiated with any domain that has finite states, deterministic transitions, and optional terminal states.

Regulatory_Instance.thy instantiates the generic safety framework with the regulatory state model. It defines the five regulatory states and seven regulatory actions, proves fundamental properties (terminal absorption, universal reachability of confiscation, transition exclusions with legal justification), models a synchronization protocol with preemptive locking, and proves the main safety theorems: *regulatory homomorphism* (cross-chain consistency after sync), *valid state preservation* (sync maintains the global validity invariant), and *multi-domain parametric instantiation*. In addition, it instantiates **state_preservation** (*escalation preservation*, see Section 1.3) and **symmetric_state_preservation** (*onchain DAML bridge*, see Section 1.4) with concrete heterogeneous-action and layer-crossing examples.

Priority_Resolution.thy defines domain-independent locales for liveness properties under Byzantine faults: **priority_system** for deterministic selection from a finite message set with injective priorities, **deadlock_free_locking** for timeout-based forced release of locks, and **fair_leader_system** for starvation freedom under periodic honest leader scheduling. The locales are parameterized over abstract types and impose no dependency on the regulatory domain; they are reusable for any distributed system requiring deterministic conflict resolution, deadlock freedom, or fair scheduling.

DQuencer_Instance.thy instantiates the generic liveness framework with the regulatory domain. It defines authority levels (REGIONAL, NATIONAL, INTERNATIONAL), a four-component priority key with proven injectivity (under well-formedness bounds), a Byzantine fault model with the BFT threshold $n \geq 3f + 1$, and instantiates each of the three liveness locales for the regulatory consensus system. The **priority_system** instantiation is performed on the **priority_key** type as carrier, with the underlying D-quencer message recovered via a unicity corollary inside a well-formedness sublocale (Section 1.5); the **deadlock_free_locking**

and `fair_leader_system` instantiations follow directly from the system’s lock timeout and leader schedule. The theory proves the main liveness consequences (*select highest deterministic*, *deadlock freedom*, *eventual completion*) and concludes with the *combined safety liveness* theorem (Section 1.6) that links the liveness results back to the synchronization function defined in `Regulatory_Instance.thy`, discharging the honest-node assumption of the safety proof under Byzantine faults.

1.1 Theory Dependency Structure

The four theories form two independent generic-instance pairs that meet at `DQuencer_Instance.thy`: `State_Preservation.thy` is imported by `Regulatory_Instance.thy`; `Priority_Resolution.thy` depends only on `Main` and is deliberately kept independent of the regulatory domain to preserve reusability; `DQuencer_Instance.thy` imports both `Priority_Resolution.thy` and `Regulatory_Instance.thy`, serving as the join point where the liveness framework is applied to the regulatory model and combined with the safety results.

1.2 Design Decisions

The regulatory state model incorporates several deliberate design decisions based on legal precedence and operational considerations:

- `RECOVER` and `LIQUIDATE` are excluded from the state machine as they involve force transfers or external `DEX` interactions rather than state transitions.
- The direct transition `SEIZED` $\rightarrow$ `FROZEN` is excluded because seizure is a strictly stronger legal constraint than freezing; relaxation requires explicit release first.
- The direct transition `FROZEN` $\rightarrow$ `RESTRICTED` is excluded to enforce passage through `ACTIVE`.
- Preemptive locking prevents concurrent regulatory actions on the same asset, *not* double-spending (which is handled at a different layer).

The liveness model adopts an assume-guarantee approach for starvation freedom: rather than directly modeling the probabilistic guarantees of VRF-based leader election (which would require extensive probability theory infrastructure), the `fair_leader_system` locale assumes that within any `fairness_bound` consecutive epochs, at least one epoch has an honest leader. Under $f < n/3$ Byzantine faults, the probability that a Byzantine leader is elected k consecutive times is $(f/n)^k$, which decreases exponentially. The fairness bound captures this as a deterministic upper bound, following standard assume-guarantee reasoning practice.

1.3 Heterogeneous-Action Instance: Escalation Preservation

The instantiation of `state_preservation` provided in `Regulatory_Instance.thy` models the situation where two chains share a regulatory state space but differ in their on-chain action vocabularies. Some jurisdictions handle de-escalation (`UNFREEZE`, `UNRESTRICT`, `RELEASE`) exclusively through separate judicial procedures rather than as on-chain actions; the corresponding chain therefore exposes only the four escalation actions (`FREEZE`, `SEIZE`, `CONFISCATE`, `RESTRICT`) on chain. We model this asymmetry by using two distinct action types: `reg_action` for the source chain (seven constructors) and a dedicated four-constructor datatype `chain_b_action` for the target chain.

The instantiation exploits two locale features simultaneously. First, the source action set parameter `actionss` is instantiated with the strict subset `{FREEZE, SEIZE, CONFISCATE, RESTRICT}` of `reg_action`, scoping structural preservation to the escalation actions only. Second, the source and target action types (`'a` vs. `'b`) carry different datatypes, exercising the locale’s heterogeneous action signature. The naturality conditions hold by case analysis on the four escalation actions and the five regulatory states, with the target chain’s transition function defined to mirror the source on the escalation subset. Sequential preservation follows as a direct corollary of the locale’s main theorem: every escalation action sequence on the source chain produces, when mapped, an equivalent sequence on the target chain ending in the same regulatory state.

This instance is more than a convenience: it isolates the locale’s `actionss` parameter as the formal handle for expressing partial action vocabularies, a concern that recurs across multi-jurisdiction protocol coordination, escalation-only consensus systems, and any cross-domain mapping where the two domains expose different on-domain actions.

1.4 Layer-Crossing Instance: Onchain DAML Bridge

The instantiation of `symmetric_state_preservation` provided in `Regulatory_Instance.thy` models the bidirectional binding between two representations of the same regulatory state: the on-chain `reg_state` enum and an off-chain DAML structured permission record (`daml_perm`). The DAML record carries a status tag together with auxiliary fields (`seized_by`, `restriction_scope`) that hold the party / scope metadata required by the corresponding regulatory status; a type-level invariant `valid_daml_perm` ties the auxiliary fields to the status tag (non-`None` exactly when the status semantically requires them).

The action vocabularies on the two layers coincide: both layers process the seven actions of `reg_action`, so the action map is the identity in both directions. All non-trivial content of the symmetric instance there-

fore lives in the layer-crossing state mapping: the bijection between the five-element `reg_state` enum and the image of `reg_to_daml` in `daml_perm`. The roundtrip assumptions of `symmetric_state_preservation` reduce to two bijection conditions, both proved by case analysis (on `reg_state` for one direction, on the status tag plus `valid_daml_perm` for the other). As a corollary, `reg_to_daml` is injective on `reg_states`, which is the formal expression of “no information loss in the layer-crossing representation”.

This is the abstraction that underlies the semantic interoperability between an on-chain settlement layer (where states are represented as enums on integer-valued storage slots) and an off-chain ledger built on DAML’s permission model. The structured-record side captures DAML’s *need-to-know* approach where party / scope metadata is bound to the entitlements implied by a regulatory status; the bijection guarantees that synchronizing in either direction is faithful.

1.5 Priority Instance: Well-Formed Messages

The `priority_system` locale assumes priority injectivity unconditionally on its carrier type. In the D-quencer setting, injectivity of `make_priority_key` holds only when the message fields satisfy two well-formedness bounds: `msg_timestamp ≤ max_time` and `dqm_source_node ≤ max_node`. These bounds correspond to the system parameters of any concrete deployment (a fixed-precision time clock and a fixed roster of nodes), but they are not properties of the underlying `dq_message` record type as such.

To bridge this gap without imposing the well-formedness bounds at the type level, we instantiate `priority_system` directly on the `priority_key` type (the lexicographic four-tuple) with the identity function as the priority projection; injectivity is then satisfied by reflexivity on the carrier. The connection back to `dq_message` is made inside a sublocale `dquencer_priority_context` extending `dquencer_system` with a fixed message set `msg_set` satisfying the bounds, which defines the priority-key set `msg_priority_keys = make_priority_key max_time max_node msg_set`.

A further sublocale `dquencer_priority_concrete` adds the priority-distinctness assumption on `msg_set` (distinct messages produce distinct priority keys, a natural BFT-consensus precondition on authority / timestamp / severity / source-node tuples). Under this assumption the priority key uniquely identifies its underlying message, captured by a definition `recover_msg:: priority_key ⇒ dq_message` together with a uniqueness lemma derived from `priority_key_injectivity` and the distinctness hypothesis.

The `dquencer_system` locale itself is not modified. Concrete deterministic-selection consequences for the D-quencer (`dq_select_highest_deterministic`, `dq_select_highest_in_set`, and the message-recovery corollary `dq_select_highest_message`) are stated inside the sublocale; together they show that running the generic `select_highest` on `msg_priority_keys` and then calling `recover_msg` yields

a deterministic choice of the highest-priority well-formed message in `msg_set`.

1.6 Combined Safety and Liveness

The `combined_safety_liveness` theorem in `DQuencer_Instance.thy` ties the synchronization function from the safety side to the liveness guarantees of the liveness side. Concretely: given a valid global state, an asset present on the source chain in a regulatory state from which a transition is enabled, and the asset not currently locked, the synchronization function returns `Some` and the resulting global state is again valid.

The structural significance of the theorem is that it discharges the honest-node assumption of the safety proof under Byzantine faults. The safety theorems are stated in a model where the synchronization function executes atomically; the liveness locales (`deadlock_free_locking` and `fair_leader_system`) formalize the conditions under which atomicity can be approximated in a Byzantine-fault-tolerant deployment with timeout-based forced lock release and a fair leader schedule. The combined theorem shows that under these liveness conditions, every legitimate sync request that clears the safety pre-conditions has a successful outcome that preserves the safety invariant.

1.7 Beyond the Atomic Sync Model

The atomic sync model is the natural locale-based abstraction for proving structural correctness of cross-domain state preservation inside a single AFP entry. Lifting the model to a partially synchronous network with message delays, redelivery, and out-of-order arrival is the next step. The locales in this entry are designed to support that lift: `state_preservation` carries the structural correctness condition that any concrete protocol must preserve; `deadlock_free_locking` and `fair_leader_system` provide the operational abstractions through which atomic sync can be realized as a sequence of bounded-duration steps. We leave the construction of the lift to subsequent entries.

```
theory State-Preservation
imports Main
begin
```

2 Generic State Machine Locale

A state machine with finite state set, deterministic partial transition function, and optional terminal states. Terminal states accept no transitions.

```
locale state-machine =
  fixes states :: 's set
  and actions :: 'a set
```

```

and transition :: 's  $\Rightarrow$  'a  $\Rightarrow$  's option
and terminal :: 's set
assumes finite-states: finite states
and finite-actions: finite actions
and terminal-subset: terminal  $\subseteq$  states
and terminal-absorbing:  $\llbracket s \in \text{terminal}; a \in \text{actions} \rrbracket \implies \text{transition } s \ a =$ 
None
and transition-closed:  $\llbracket s \in \text{states}; a \in \text{actions}; \text{transition } s \ a = \text{Some } s' \rrbracket \implies$ 
s'  $\in$  states
and transition-domain:  $\llbracket s \notin \text{states} \rrbracket \implies \text{transition } s \ a = \text{None}$ 
begin

```

Determinism is inherent: transition is a function, not a relation.

lemma *transition-deterministic*:

```

transition s a = Some s1  $\implies$  transition s a = Some s2  $\implies$  s1 = s2
by simp

```

Sequential composition of actions (partial, returns None on first failure).

```

fun apply-actions :: 's  $\Rightarrow$  'a list  $\Rightarrow$  's option where
  apply-actions s [] = Some s
| apply-actions s (a # as) = (case transition s a of
  None  $\Rightarrow$  None
| Some s'  $\Rightarrow$  apply-actions s' as)

```

lemma *apply-actions-closed*:

```

 $\llbracket s \in \text{states}; \forall a \in \text{set } \text{as}. a \in \text{actions}; \text{apply-actions } s \ \text{as} = \text{Some } s' \rrbracket$ 
 $\implies s' \in \text{states}$ 
by (induction as arbitrary: s) (auto split: option.splits dest: transition-closed)

```

Note: *apply-actions* ?*s* [] = *Some* ?*s* already provides the simp rule *apply-actions* *s* [] = *Some* *s*, so no separate [simp] lemma is needed.

lemma *apply-actions-terminal*:

```

 $\llbracket s \in \text{terminal}; a \in \text{actions} \rrbracket \implies \text{apply-actions } s \ (a \ \# \ \text{as}) = \text{None}$ 
by (simp add: terminal-absorbing)

```

end

3 Cross-Domain State Preservation Locale

Two state machines connected by a synchronization map. The key property (naturality / homomorphism) states that synchronizing after a local transition yields the same result as transitioning after synchronization.

More precisely: for state machines (*S_src*, *T_src*) and (*S_tgt*, *T_tgt*) connected by sync (mapping source states to target states), the following diagram commutes:

```

  s_src -T_src(a)-> s_src' | | sync sync | | v v s_tgt -T_tgt(a)->
s_tgt'

```

That is: $\text{sync } (\text{T_src } s \ a) = \text{T_tgt } (\text{sync } s) \ a$

This is the structural preservation guarantee: a transition performed at the source domain, when synchronized, produces the same result as performing the corresponding transition at the target domain.

```

locale state-preservation =
  source: state-machine statess actionss transitions terminals +
  target: state-machine statest actionst transitiont terminalt
  for statess :: 's set and actionss :: 'a set
    and transitions :: 's ⇒ 'a ⇒ 's option
    and terminals :: 's set
    and statest :: 't set and actionst :: 'b set
    and transitiont :: 't ⇒ 'b ⇒ 't option
    and terminalt :: 't set +
  fixes state-map :: 's ⇒ 't
    and action-map :: 'a ⇒ 'b
  assumes state-map-well-defined: s ∈ statess ⇒ state-map s ∈ statest
    and action-map-well-defined: a ∈ actionss ⇒ action-map a ∈ actionst
    and terminal-preservation: s ∈ terminals ⇒ state-map s ∈ terminalt
  — The naturality / homomorphism condition:
  and naturality:
    [| s ∈ statess; a ∈ actionss; transitions s a = Some s' |]
    ⇒ transitiont (state-map s) (action-map a) = Some (state-map s')
  and naturality-none:
    [| s ∈ statess; a ∈ actionss; transitions s a = None |]
    ⇒ transitiont (state-map s) (action-map a) = None
begin

```

The fundamental theorem: state preservation extends to sequences of actions. If a sequence of transitions is valid at the source, the mapped sequence is valid at the target with the mapped final state.

theorem sequential-preservation:

```

  assumes s ∈ statess
    and ∀ a ∈ set as. a ∈ actionss
    and source.apply-actions s as = Some s'
  shows target.apply-actions (state-map s) (map action-map as) = Some (state-map s')
  using assms
proof (induction as arbitrary: s)
  case Nil
  then show ?case by simp
next
  case (Cons a as)
  then obtain s-mid where
    step: transitions s a = Some s-mid and
    rest: source.apply-actions s-mid as = Some s'
  by (auto split: option.splits)
from Cons.prem1 Cons.prem2 step have s-mid ∈ statess
  using source.transition-closed by auto

```

```

from Cons.premis(1) Cons.premis(2) step
have mapped-step: transitiont (state-map s) (action-map a) = Some (state-map
s-mid)
  using naturality by auto
from ⟨s-mid ∈ statess⟩ Cons.premis(2) rest
have target.apply-actions (state-map s-mid) (map action-map as) = Some (state-map
s')
  using Cons.IH by auto
with mapped-step show ?case by simp
qed

```

theorem sequential-preservation-none:

```

assumes s ∈ statess
and ∀ a ∈ set as. a ∈ actionss
and source.apply-actions s as = None
shows target.apply-actions (state-map s) (map action-map as) = None
using assms
proof (induction as arbitrary: s)
  case Nil
  then show ?case by simp
next
  case (Cons a as)
  show ?case
  proof (cases transitions s a)
    case None
    then have transitiont (state-map s) (action-map a) = None
      using naturality-none Cons.premis(1) Cons.premis(2) by auto
    then show ?thesis by simp
  next
    case (Some s-mid)
    then have mid-in: s-mid ∈ statess
      using source.transition-closed Cons.premis(1) Cons.premis(2) by auto
    from Some Cons.premis(3) have source.apply-actions s-mid as = None
      by simp
    then have ih: target.apply-actions (state-map s-mid) (map action-map as) =
None
      using Cons.IH mid-in Cons.premis(2) by auto
    from Some have transitiont (state-map s) (action-map a) = Some (state-map
s-mid)
      using naturality Cons.premis(1) Cons.premis(2) by auto
    with ih show ?thesis by simp
  qed
qed

```

Terminal states are preserved: if a source state is terminal, its image is terminal.

corollary terminal-image-absorbing:

```

assumes s ∈ terminals and a ∈ actionss
shows transitiont (state-map s) (action-map a) = None

```

```

proof –
  from assms(1) have  $s \in \text{states}_s$  using source.terminal-subset by auto
  from assms have  $\text{transition}_s\ s\ a = \text{None}$ 
  using source.terminal-absorbing by auto
  then show ?thesis
  using naturality-none  $\langle s \in \text{states}_s \rangle$  assms(2) by auto
qed

end

```

4 Symmetric State Preservation

For bidirectional synchronization, both directions must preserve structure. This locale composes two **state_preservation** instances with inverse maps, establishing that synchronization in either direction is structure-preserving.

```

locale symmetric-state-preservation =
  forward: state-preservation
     $\text{states}_s\ \text{actions}_s\ \text{transition}_s\ \text{terminal}_s$ 
     $\text{states}_t\ \text{actions}_t\ \text{transition}_t\ \text{terminal}_t$ 
    state-map action-map +
  backward: state-preservation
     $\text{states}_t\ \text{actions}_t\ \text{transition}_t\ \text{terminal}_t$ 
     $\text{states}_s\ \text{actions}_s\ \text{transition}_s\ \text{terminal}_s$ 
    state-map-inv action-map-inv
  for  $\text{states}_s :: 's\ \text{set}$  and  $\text{actions}_s :: 'a\ \text{set}$ 
  and  $\text{transition}_s :: 's \Rightarrow 'a \Rightarrow 's\ \text{option}$ 
  and  $\text{terminal}_s :: 's\ \text{set}$ 
  and  $\text{states}_t :: 't\ \text{set}$  and  $\text{actions}_t :: 'b\ \text{set}$ 
  and  $\text{transition}_t :: 't \Rightarrow 'b \Rightarrow 't\ \text{option}$ 
  and  $\text{terminal}_t :: 't\ \text{set}$ 
  and  $\text{state-map} :: 's \Rightarrow 't$  and  $\text{action-map} :: 'a \Rightarrow 'b$ 
  and  $\text{state-map-inv} :: 't \Rightarrow 's$  and  $\text{action-map-inv} :: 'b \Rightarrow 'a$  +
  assumes roundtrip-state-src:  $s \in \text{states}_s \implies \text{state-map-inv} (\text{state-map}\ s) = s$ 
  and roundtrip-state-tgt:  $t \in \text{states}_t \implies \text{state-map} (\text{state-map-inv}\ t) = t$ 
  and roundtrip-action-src:  $a \in \text{actions}_s \implies \text{action-map-inv} (\text{action-map}\ a) = a$ 
  and roundtrip-action-tgt:  $b \in \text{actions}_t \implies \text{action-map} (\text{action-map-inv}\ b) = b$ 
begin

```

Under symmetric preservation with roundtrip maps, **state_map** is injective on the state set – distinct source states map to distinct target states. This guarantees no information loss during synchronization.

```

lemma state-map-injective:
   $\llbracket s1 \in \text{states}_s; s2 \in \text{states}_s; \text{state-map}\ s1 = \text{state-map}\ s2 \rrbracket \implies s1 = s2$ 
  using roundtrip-state-src by metis

end

```

5 Multi-Domain State Preservation

Generalization to N domains sharing the same abstract state machine. This models scenarios where a single asset exists on multiple domains (e.g., blockchain networks), all of which must reflect the same state.

The key property: if one domain transitions, the synchronization brings ALL other domains to the same resulting state.

We model this as: all domains are instances of the same abstract state machine, connected by identity-like state/action maps. The "same abstract state machine" assumption holds when the protocol enforces a uniform state model across all domains.

```

locale multi-domain-preservation =
  fixes domains :: 'd set
    and states :: 's set
    and actions :: 'a set
    and transition :: 's  $\Rightarrow$  'a  $\Rightarrow$  's option
    and terminal :: 's set
    and domain-state :: 'd  $\Rightarrow$  'id  $\Rightarrow$  's option
    — Current state of asset 'id in domain 'd
  assumes fin-domains: finite domains
    and sm: state-machine states actions transition terminal
    and consistent-init:
    — Precondition: all domains holding an asset agree on its state
     $\llbracket d1 \in \text{domains}; d2 \in \text{domains};$ 
      domain-state d1 aid = Some s1; domain-state d2 aid = Some s2  $\rrbracket$ 
     $\implies s1 = s2$ 
begin

```

Which domains hold a given asset.

```

definition connected-domains :: 'id  $\Rightarrow$  'd set where
  connected-domains aid = {d  $\in$  domains. domain-state d aid  $\neq$  None}

```

The global consensus state of an asset (well-defined by `consistent_init`).

```

definition consensus-state :: 'id  $\Rightarrow$  's option where
  consensus-state aid =
    (if connected-domains aid = {} then None
     else domain-state (SOME d. d  $\in$  connected-domains aid) aid)

```

After synchronizing an action on an asset, all connected domains reflect the new state. This is the cross-domain consistency theorem.

```

definition sync-all ::
  'd  $\Rightarrow$  'a  $\Rightarrow$  'id  $\Rightarrow$  ('d  $\Rightarrow$  'id  $\Rightarrow$  's option)  $\Rightarrow$  ('d  $\Rightarrow$  'id  $\Rightarrow$  's option) option
where
  sync-all source action aid ds =
    (case ds source aid of
      None  $\Rightarrow$  None
    | Some s  $\Rightarrow$ 
      (case transition s action of

```

```

      None  $\Rightarrow$  None
    | Some s'  $\Rightarrow$ 
      Some ( $\lambda d$  aid'.
        if aid' = aid  $\wedge$  d  $\in$  connected-domains aid
        then Some s'
        else ds d aid'))))

```

The cross-domain consistency theorem: after **sync_all**, every connected domain agrees on the new state.

```

theorem cross-domain-consistency:
  assumes source  $\in$  domains
    and domain-state source aid = Some s
    and s  $\in$  states
    and action  $\in$  actions
    and transition s action = Some s'
    and sync-all source action aid domain-state = Some ds'
    and d  $\in$  connected-domains aid
  shows ds' d aid = Some s'
proof -
  from assms(2,5,6) have
    ds' = ( $\lambda d$  aid'.
      if aid' = aid  $\wedge$  d  $\in$  connected-domains aid
      then Some s'
      else domain-state d aid')
    unfolding sync-all-def by auto
  with assms(7) show ?thesis by simp
qed

```

sync_all does not affect other assets.

```

theorem sync-isolation:
  assumes sync-all source action aid domain-state = Some ds'
    and aid'  $\neq$  aid
  shows ds' d aid' = domain-state d aid'
proof -
  from assms(1) obtain s s' where
    domain-state source aid = Some s
    transition s action = Some s'
    unfolding sync-all-def by (auto split: option.splits)
  then have ds' = ( $\lambda d$  aid'.
    if aid' = aid  $\wedge$  d  $\in$  connected-domains aid
    then Some s'
    else domain-state d aid')
    using assms(1) unfolding sync-all-def by auto
  with assms(2) show ?thesis by auto
qed

end

end

```

```

theory Regulatory-Instance
  imports State-Preservation
begin

```

6 Regulatory State and Action Definitions

```

datatype reg-state = ACTIVE | FROZEN | SEIZED | CONFISCATED | RE-
STRICTED

```

```

datatype reg-action = FREEZE | SEIZE | CONFISCATE | RESTRICT
  | UNFREEZE | UNRESTRICT | RELEASE

```

The regulatory transition function. Partial: returns None for invalid (state, action) combinations. The transition table encodes legal precedence: seizure is stronger than freezing, confiscation is terminal and universally reachable, and intermediate states must return to Active before lateral transitions.

```

fun reg-transition :: reg-state  $\Rightarrow$  reg-action  $\Rightarrow$  reg-state option where
  — From ACTIVE: all forward actions valid
  | reg-transition ACTIVE FREEZE      = Some FROZEN
  | reg-transition ACTIVE SEIZE       = Some SEIZED
  | reg-transition ACTIVE CONFISCATE = Some CONFISCATED
  | reg-transition ACTIVE RESTRICT   = Some RESTRICTED
  | reg-transition ACTIVE UNFREEZE   = None
  | reg-transition ACTIVE UNRESTRICT = None
  | reg-transition ACTIVE RELEASE    = None
  — From FROZEN: unfreeze, escalate to SEIZED or CONFISCATED
  | reg-transition FROZEN UNFREEZE   = Some ACTIVE
  | reg-transition FROZEN SEIZE      = Some SEIZED
  | reg-transition FROZEN CONFISCATE = Some CONFISCATED
  | reg-transition FROZEN FREEZE     = None
  | reg-transition FROZEN RESTRICT   = None
  | reg-transition FROZEN UNRESTRICT = None
  | reg-transition FROZEN RELEASE    = None
  — From SEIZED: release or final confiscation only
  | reg-transition SEIZED RELEASE    = Some ACTIVE
  | reg-transition SEIZED CONFISCATE = Some CONFISCATED
  | reg-transition SEIZED FREEZE     = None
  | reg-transition SEIZED SEIZE      = None
  | reg-transition SEIZED RESTRICT   = None
  | reg-transition SEIZED UNFREEZE   = None
  | reg-transition SEIZED UNRESTRICT = None
  — From CONFISCATED: terminal no transitions out
  | reg-transition CONFISCATED FREEZE   = None
  | reg-transition CONFISCATED SEIZE    = None
  | reg-transition CONFISCATED CONFISCATE = None

```


	<i>reg-transition</i>	<i>CONFISCATED RESTRICT</i>	<i>= None</i>
	<i>reg-transition</i>	<i>CONFISCATED UNFREEZE</i>	<i>= None</i>
	<i>reg-transition</i>	<i>CONFISCATED UNRESTRICT</i>	<i>= None</i>
	<i>reg-transition</i>	<i>CONFISCATED RELEASE</i>	<i>= None</i>
	— From <i>RESTRICTED</i> : <i>unrestrict</i> , <i>escalate</i> to <i>FROZEN</i> or <i>CONFISCATED</i>		
	<i>reg-transition</i>	<i>RESTRICTED UNRESTRICT</i>	<i>= Some ACTIVE</i>
	<i>reg-transition</i>	<i>RESTRICTED FREEZE</i>	<i>= Some FROZEN</i>
	<i>reg-transition</i>	<i>RESTRICTED CONFISCATE</i>	<i>= Some CONFISCATED</i>
	<i>reg-transition</i>	<i>RESTRICTED RESTRICT</i>	<i>= None</i>
	<i>reg-transition</i>	<i>RESTRICTED SEIZE</i>	<i>= None</i>
	<i>reg-transition</i>	<i>RESTRICTED UNFREEZE</i>	<i>= None</i>
	<i>reg-transition</i>	<i>RESTRICTED RELEASE</i>	<i>= None</i>

7 Fundamental Properties of the Transition Function

7.1 Invariant I1: *CONFISCATED* is terminal

lemma *confiscated-terminal*:

reg-transition CONFISCATED a = None
by (*cases a*) *auto*

7.2 Invariant I2: *CONFISCATE* reaches *CONFISCATED* from any non-terminal state

lemma *confiscate-universal*:

s ≠ CONFISCATED ⇒ reg-transition s CONFISCATE = Some CONFISCATED
by (*cases s*) *auto*

7.3 Invariant I3: Determinism (inherent from function definition)

Determinism follows directly from the **fun** definition.

7.4 SEIZED to FROZEN exclusion

SEIZED is a strictly stronger constraint than *FROZEN*. Direct transition from *SEIZED* to *FROZEN* is legally nonsensical (cannot "weaken" a court-ordered seizure to a mere freeze). The path is: *RELEASE* then *ACTIVE* then *FREEZE*.

lemma *seized-no-freeze*:

reg-transition SEIZED FREEZE = None
by *simp*

lemma *seized-no-unfreeze*:

reg-transition SEIZED UNFREEZE = None

by *simp*

lemma *seized-no-restrict:*

reg-transition SEIZED RESTRICT = None

by *simp*

7.5 FROZEN to RESTRICTED exclusion

Must go through ACTIVE: UNFREEZE then ACTIVE then RESTRICT.

lemma *frozen-no-restrict:*

reg-transition FROZEN RESTRICT = None

by *simp*

7.6 Transition completeness: every non-terminal state has at least one valid action

lemma *non-terminal-has-action:*

assumes $s \neq \text{CONFISCATED}$

shows $\exists a. \text{reg-transition } s \ a \neq \text{None}$

proof (*cases s*)

case *ACTIVE*

show *?thesis*

apply (*rule exI[where x=FREEZE]*)

using *ACTIVE* by *simp*

next

case *FROZEN*

show *?thesis*

apply (*rule exI[where x=UNFREEZE]*)

using *FROZEN* by *simp*

next

case *SEIZED*

show *?thesis*

apply (*rule exI[where x=RELEASE]*)

using *SEIZED* by *simp*

next

case *CONFISCATED*

then show *?thesis* using *assms* by *contradiction*

next

case *RESTRICTED*

show *?thesis*

apply (*rule exI[where x=UNRESTRICT]*)

using *RESTRICTED* by *simp*

qed

7.7 Reachability from ACTIVE

Every state is reachable from ACTIVE via some sequence of actions.

lemma *active-reaches-frozen: reg-transition ACTIVE FREEZE = Some FROZEN*

by *simp*

lemma *active-reaches-seized*: *reg-transition ACTIVE SEIZE = Some SEIZED*
by *simp*

lemma *active-reaches-confiscated*: *reg-transition ACTIVE CONFISCATE = Some CONFISCATED*
by *simp*

lemma *active-reaches-restricted*: *reg-transition ACTIVE RESTRICT = Some RESTRICTED*
by *simp*

7.8 Return paths to ACTIVE

lemma *frozen-returns*: *reg-transition FROZEN UNFREEZE = Some ACTIVE*
by *simp*

lemma *seized-returns*: *reg-transition SEIZED RELEASE = Some ACTIVE*
by *simp*

lemma *restricted-returns*: *reg-transition RESTRICTED UNRESTRICT = Some ACTIVE*
by *simp*

8 State Machine Instance

We show that `reg_state` / `reg_action` / `reg_transition` satisfy the `state_machine` locale assumptions.

definition *reg-states* :: *reg-state set* **where**
reg-states = {*ACTIVE*, *FROZEN*, *SEIZED*, *CONFISCATED*, *RESTRICTED*}

definition *reg-actions* :: *reg-action set* **where**
reg-actions = {*FREEZE*, *SEIZE*, *CONFISCATE*, *RESTRICT*, *UNFREEZE*, *UNRESTRICT*, *RELEASE*}

definition *reg-terminal* :: *reg-state set* **where**
reg-terminal = {*CONFISCATED*}

lemma *reg-states-UNIV*: *reg-states = UNIV*
unfolding *reg-states-def* **by** (*auto*, *case-tac x*, *auto*)

lemma *reg-actions-UNIV*: *reg-actions = UNIV*
unfolding *reg-actions-def* **by** (*auto*, *case-tac x*, *auto*)

lemma *reg-transition-closed*:
[*reg-transition s a = Some s'*] $\implies s' \in \text{reg-states}$
unfolding *reg-states-def* **by** (*cases s*; *cases a*; *auto*)

```

interpretation reg-sm: state-machine reg-states reg-actions reg-transition reg-terminal
proof unfold-locales
  show finite reg-states unfolding reg-states-def by auto
next
  show finite reg-actions unfolding reg-actions-def by auto
next
  show reg-terminal  $\subseteq$  reg-states unfolding reg-terminal-def reg-states-def by auto
next
  fix s a
  assume s  $\in$  reg-terminal a  $\in$  reg-actions
  then show reg-transition s a = None
    unfolding reg-terminal-def by (auto simp: confiscated-terminal)
next
  fix s a s'
  assume s  $\in$  reg-states a  $\in$  reg-actions reg-transition s a = Some s'
  then show s'  $\in$  reg-states
    using reg-transition-closed by auto
next
  fix s :: reg-state and a :: reg-action
  assume s  $\notin$  reg-states
  then show reg-transition s a = None
    using reg-states-UNIV by auto
qed

```

Bundle form of the regulatory state machine, used as a structural unit when discharging **state_preservation** obligations whose source or target matches the full **reg_actions** vocabulary. Mirrors the existing **chain_b_state_machine** and **daml_state_machine** bundles defined later in the theory.

```

lemma reg-state-machine:
  state-machine reg-states reg-actions reg-transition reg-terminal
  using reg-sm.finite-states reg-sm.finite-actions reg-sm.terminal-subset
    reg-sm.terminal-absorbing reg-sm.transition-closed reg-sm.transition-domain
  by (rule state-machine.intro)

```

9 Asset and Global State Modeling

```

type-synonym asset-id = nat
type-synonym chain-id = nat
type-synonym authority-id = nat
type-synonym timestamp = nat

record asset-state =
  as-asset-id :: asset-id
  as-reg-state :: reg-state
  as-owner    :: nat      — Abstract address
  as-locked   :: bool     — Preemptive lock

```

type-synonym *chain-state* = *asset-id* $\Rightarrow$ *asset-state option*

record *global-state* =
 gs-chains :: *chain-id* $\Rightarrow$ *chain-state*
 gs-locks :: *asset-id* $\Rightarrow$ *bool*

record *oss-message* =
 msg-action :: *reg-action*
 msg-asset-id :: *asset-id*
 msg-authority :: *authority-id*
 msg-timestamp :: *timestamp*
 msg-source :: *chain-id*
 msg-targets :: *chain-id set*

10 State Accessors and Predicates

definition *get-asset-state* :: *global-state* $\Rightarrow$ *chain-id* $\Rightarrow$ *asset-id* $\Rightarrow$ *asset-state option* **where**
 get-asset-state *gs* *cid* *aid* = *gs-chains* *gs* *cid* *aid*

definition *get-reg-state* :: *global-state* $\Rightarrow$ *chain-id* $\Rightarrow$ *asset-id* $\Rightarrow$ *reg-state option* **where**
 get-reg-state *gs* *cid* *aid* =
 (case *get-asset-state* *gs* *cid* *aid* of
 None $\Rightarrow$ None
 | Some *ast* $\Rightarrow$ Some (*as-reg-state* *ast*))

definition *asset-exists* :: *global-state* $\Rightarrow$ *chain-id* $\Rightarrow$ *asset-id* $\Rightarrow$ *bool* **where**
 asset-exists *gs* *cid* *aid* = (*get-asset-state* *gs* *cid* *aid* $\neq$ None)

definition *is-locked* :: *global-state* $\Rightarrow$ *asset-id* $\Rightarrow$ *bool* **where**
 is-locked *gs* *aid* = *gs-locks* *gs* *aid*

Connected chains: all chains holding a given asset.

definition *connected-chains* :: *global-state* $\Rightarrow$ *asset-id* $\Rightarrow$ *chain-id set* **where**
 connected-chains *gs* *aid* = {*cid*. *asset-exists* *gs* *cid* *aid*}

11 Global State Validity

A global state is valid if all chains holding the same asset agree on its regulatory state. This is the cross-chain consistency invariant that synchronization must preserve.

definition *consistent-state* :: *global-state* $\Rightarrow$ *bool* **where**
 consistent-state *gs* $\equiv$
 $\forall c1\ c2\ aid\ s1\ s2.$
 get-reg-state *gs* *c1* *aid* = Some *s1* $\longrightarrow$
 get-reg-state *gs* *c2* *aid* = Some *s2* $\longrightarrow$

$$s1 = s2$$

definition *no-locked-without-reason* :: *global-state* $\Rightarrow$ *bool* **where**

no-locked-without-reason *gs* $\equiv$

$\forall aid. \neg is_locked\ gs\ aid$

— In a quiescent valid state, no assets are locked. Locks are transient, held only during sync operations.

definition *valid-state* :: *global-state* $\Rightarrow$ *bool* **where**

valid-state *gs* $\equiv consistent_state\ gs \wedge no_locked_without_reason\ gs$

12 Synchronization Protocol

12.1 Lock acquisition

definition *acquire-lock* :: *global-state* $\Rightarrow$ *asset-id* $\Rightarrow$ *global-state option* **where**

acquire-lock *gs* *aid* =

(if *is-locked* *gs* *aid* then *None*

else *Some* (*gs* | *gs*-locks := (*gs*-locks *gs*)(*aid* := *True*) |))

definition *release-lock* :: *global-state* $\Rightarrow$ *asset-id* $\Rightarrow$ *global-state* **where**

release-lock *gs* *aid* = *gs* | *gs*-locks := (*gs*-locks *gs*)(*aid* := *False*) |

12.2 State update across chains

Update the regulatory state of an asset on a specific chain. Preserves all other asset fields (owner, etc.).

definition *update-chain-reg-state* ::

global-state $\Rightarrow$ *chain-id* $\Rightarrow$ *asset-id* $\Rightarrow$ *reg-state* $\Rightarrow$ *global-state*

where

update-chain-reg-state *gs* *cid* *aid* *new-st* =

(let *old-chain* = *gs*-chains *gs* *cid* in

let *new-chain* = ($\lambda aid'$.

if *aid'* = *aid* then

(case *old-chain* *aid* of

None $\Rightarrow$ *None*

| *Some* *ast* $\Rightarrow$ *Some* (*ast* | *as-reg-state* := *new-st* |))

else *old-chain* *aid'*)

in *gs* | *gs*-chains := (*gs*-chains *gs*)(*cid* := *new-chain*) |)

Update all connected chains to a new regulatory state.

We define this directly rather than using `Finite_Set.fold`, because the update for a given asset on each chain is independent we simply construct a new global state where every chain's view of the asset is updated. This avoids the need for `comp_fun_commute` proofs.

definition *update-all-chains* ::

global-state $\Rightarrow$ *asset-id* $\Rightarrow$ *reg-state* $\Rightarrow$ *chain-id set* $\Rightarrow$ *global-state*

where

```

update-all-chains gs aid new-st targets =
  gs | gs-chains := (λcid.
    if cid ∈ targets then
      (λaid'. if aid' = aid then
        (case gs-chains gs cid aid of
          None ⇒ None
          | Some ast ⇒ Some (ast | as-reg-state := new-st |)))
      else gs-chains gs cid aid')
    else gs-chains gs cid) |

```

12.3 The synchronization function

sync: the core synchronization operation.

Given a source chain, action, and **asset_id**: 1. Verify the asset exists on the source chain 2. Check the transition is valid 3. Acquire global lock (fails if already locked) 4. Update all connected chains to the new state 5. Release lock

Returns None if any step fails (invalid transition, lock contention, etc.)

definition *sync* ::

chain-id ⇒ *reg-action* ⇒ *asset-id* ⇒ *global-state* ⇒ *global-state option*

where

```

sync source action aid gs =
  (case get-reg-state gs source aid of
    None ⇒ None
    | Some current-st ⇒
      (case reg-transition current-st action of
        None ⇒ None
        | Some new-st ⇒
          (case acquire-lock gs aid of
            None ⇒ None
            | Some gs-locked ⇒
              let targets = connected-chains gs aid in
              let gs-updated = update-all-chains gs-locked aid new-st targets in
              Some (release-lock gs-updated aid))))

```

13 Synchronization Properties

13.1 Idempotency

After synchronization, applying the same action again yields None (the state has already transitioned, so the same transition from the new state is typically invalid) or is a no-op.

More precisely: if sync succeeds and produces **new_st**, then **reg_transition new_st action** = None for most action/state pairs (the transition table has no self-loops).

lemma *no-self-loops*:

reg-transition s a = *Some s* ⇒ *False*

by (*cases s; cases a; auto*)

lemma *transition-changes-state*:
reg-transition s a = Some s' $\implies$ s $\neq$ s'
using *no-self-loops* **by** *auto*

13.2 Sync isolation: other assets are not affected

Synchronization on asset *aid* does not change the state of any other asset (where the asset identifier differs) on any chain.

lemma *update-chain-other-asset*:
assumes *aid' $\neq$ aid*
shows *gs-chains (update-chain-reg-state gs cid aid new-st) cid' aid' =*
gs-chains gs cid' aid'
proof (*cases cid' = cid*)
case *True*
then show *?thesis*
unfolding *update-chain-reg-state-def Let-def*
using *assms* **by** *auto*
next
case *False*
then show *?thesis*
unfolding *update-chain-reg-state-def Let-def* **by** *auto*
qed

13.3 Sync correctness: source chain updated

After *update_chain_reg_state*, the target chain reflects the new state.

lemma *update-chain-same-asset*:
assumes *gs-chains gs cid aid = Some ast*
shows *gs-chains (update-chain-reg-state gs cid aid new-st) cid aid =*
Some (ast $\sqcap$ as-reg-state := new-st $\sqcap$)
unfolding *update-chain-reg-state-def Let-def*
using *assms* **by** *auto*

13.4 Properties of *update_all_chains*

update_all_chains correctly updates any chain in the target set.

lemma *update-all-chains-in-targets*:
assumes *cid $\in$ targets*
and *gs-chains gs cid aid = Some ast*
shows *gs-chains (update-all-chains gs aid new-st targets) cid aid =*
Some (ast $\sqcap$ as-reg-state := new-st $\sqcap$)
unfolding *update-all-chains-def* **using** *assms* **by** *auto*

lemma *update-all-chains-in-targets-none*:
assumes *cid $\in$ targets*
and *gs-chains gs cid aid = None*

shows *gs-chains* (*update-all-chains gs aid new-st targets*) *cid aid* = *None*
unfolding *update-all-chains-def* **using** *assms* **by** *auto*

lemma *update-all-chains-outside-targets*:
assumes *cid* $\notin$ *targets*
shows *gs-chains* (*update-all-chains gs aid new-st targets*) *cid* = *gs-chains gs cid*
unfolding *update-all-chains-def* **using** *assms* **by** *auto*

lemma *update-all-chains-other-asset*:
assumes *aid'* $\neq$ *aid*
shows *gs-chains* (*update-all-chains gs aid new-st targets*) *cid aid'* =
gs-chains gs cid aid'
unfolding *update-all-chains-def* **using** *assms* **by** *auto*

lemma *update-all-chains-locks*:
gs-locks (*update-all-chains gs aid new-st targets*) = *gs-locks gs*
unfolding *update-all-chains-def* **by** *auto*

Key lemma: *get_reg_state* after *update_all_chains* for a chain in *targets*.

lemma *update-all-chains-reg-state*:
assumes *cid* $\in$ *targets*
and *get-asset-state gs cid aid* = *Some ast*
shows *get-reg-state* (*update-all-chains gs aid new-st targets*) *cid aid* = *Some new-st*
proof –
from *assms*(2) **have** *chain*: *gs-chains gs cid aid* = *Some ast*
unfolding *get-asset-state-def* **by** *simp*
then have *gs-chains* (*update-all-chains gs aid new-st targets*) *cid aid* =
Some (ast(| as-reg-state := new-st |))
using *update-all-chains-in-targets[OF assms(1)]* **by** *simp*
then show *?thesis*
unfolding *get-reg-state-def* *get-asset-state-def* **by** *simp*
qed

13.5 Lock mutual exclusion

If a lock is held, concurrent sync on the same asset fails. This prevents concurrent regulatory action conflicts.

lemma *lock-mutual-exclusion*:
assumes *is-locked gs aid*
shows *acquire-lock gs aid* = *None*
unfolding *acquire-lock-def* **using** *assms* **by** *auto*

lemma *lock-acquire-success*:
assumes $\neg$ *is-locked gs aid*
shows $\exists$ *gs'*. *acquire-lock gs aid* = *Some gs' $\wedge$ is-locked gs' aid*
proof –
from *assms* **have** $\neg$ *gs-locks gs aid*

```

    unfolding is-locked-def by simp
  then have acquire-lock gs aid = Some (gs | gs-locks := (gs-locks gs)(aid := True)
  |)
    unfolding acquire-lock-def is-locked-def by simp
  moreover have is-locked (gs | gs-locks := (gs-locks gs)(aid := True) |) aid
    unfolding is-locked-def by simp
  ultimately show ?thesis by auto
qed

```

```

lemma lock-release:
  shows  $\neg$  is-locked (release-lock gs aid) aid
  unfolding release-lock-def is-locked-def by auto

```

14 Main Theorem: Cross-Chain Regulatory Consistency

The central theorem of this theory. Under a valid global state (all chains consistent, no outstanding locks), if a regulatory action is applied via sync, the resulting state is also valid (all chains consistent) and every connected chain reflects the new regulatory state.

This is the regulatory instantiation of the cross-domain consistency theorem from `State_Preservation.thy`.

The proof proceeds by: 1. Unfolding sync into lock, then `update_all_chains`, then unlock 2. Showing `acquire_lock` and `release_lock` preserve chain state 3. Showing `update_all_chains` updates every chain in the target set 4. Combining these facts to establish the conclusion

Auxiliary: update on a single chain preserves consistency for other assets.

```

lemma update-chain-preserves-other-consistency:
  assumes consistent-state gs
  and aid'  $\neq$  aid
  and get-reg-state gs c1 aid' = Some s1
  and get-reg-state gs c2 aid' = Some s2
  shows get-reg-state (update-chain-reg-state gs cid aid new-st) c1 aid' = Some s1
     $\wedge$  get-reg-state (update-chain-reg-state gs cid aid new-st) c2 aid' = Some s2
  using assms unfolding get-reg-state-def get-asset-state-def
    update-chain-reg-state-def Let-def consistent-state-def
  by auto

```

Auxiliary: `release_lock` does not change chains.

```

lemma release-lock-chains:
  gs-chains (release-lock gs aid) = gs-chains gs
  unfolding release-lock-def by simp

```

```

lemma release-lock-reg-state:
  get-reg-state (release-lock gs aid) cid aid' = get-reg-state gs cid aid'
  unfolding get-reg-state-def get-asset-state-def release-lock-chains by simp

```

Auxiliary: `acquire_lock` does not change chains.

lemma *acquire-lock-chains*:
assumes *acquire-lock gs aid = Some gs'*
shows *gs-chains gs' = gs-chains gs*
using *assms* **unfolding** *acquire-lock-def is-locked-def*
by (*auto split: if-splits*)

lemma *acquire-lock-reg-state*:
assumes *acquire-lock gs aid = Some gs'*
shows *get-reg-state gs' cid aid' = get-reg-state gs cid aid'*
using *acquire-lock-chains[OF assms]*
unfolding *get-reg-state-def get-asset-state-def* **by** *simp*

Auxiliary: `connected_chains` uses original gs, not locked gs.

lemma *acquire-lock-asset-exists*:
assumes *acquire-lock gs aid = Some gs'*
shows *asset-exists gs' cid aid' = asset-exists gs cid aid'*
using *acquire-lock-chains[OF assms]*
unfolding *asset-exists-def get-asset-state-def* **by** *simp*

The cross-chain regulatory homomorphism theorem.

theorem *regulatory-homomorphism*:
assumes *valid: valid-state gs*
and *exists: asset-exists gs source aid*
and *current: get-reg-state gs source aid = Some s*
and *trans: reg-transition s action = Some s'*
and *synced: sync source action aid gs = Some gs'*
and *connected: c ∈ connected-chains gs aid*
and *fin: finite (connected-chains gs aid)*
shows *get-reg-state gs' c aid = Some s'*
proof —
— Step 1: Unfold sync to extract intermediate states
from *synced current trans*
obtain *gs-locked* **where**
lock: acquire-lock gs aid = Some gs-locked
and *gs'-def: gs' = release-lock*
(update-all-chains gs-locked aid s' (connected-chains gs aid)) aid
unfolding *sync-def*
by (*auto split: option.splits simp: Let-def*)
— Step 2: `acquire_lock` preserves chains
have *locked-chains: gs-chains gs-locked = gs-chains gs*
using *acquire-lock-chains[OF lock]* **by** *simp*
— Step 3: *c* is in `connected_chains`, so the asset exists on *c*
from *connected* **have** *asset-exists gs c aid*
unfolding *connected-chains-def* **by** *simp*
then obtain *ast-c* **where** *ast-c: gs-chains gs c aid = Some ast-c*
unfolding *asset-exists-def get-asset-state-def* **by** *auto*

— Step 4: Therefore asset also exists in `gs_locked` on chain `c`
have *ast-locked*: *gs-chains gs-locked c aid = Some ast-c*
using *ast-c locked-chains* **by** *simp*

— Step 5: `update_all_chains` updates chain `c`
have *updated*: *gs-chains (update-all-chains gs-locked aid s' (connected-chains gs aid)) c aid =*
Some (ast-c | as-reg-state := s' |)
using *update-all-chains-in-targets[OF connected ast-locked]* **by** *simp*

— Step 6: `release_lock` does not change chains
have *gs-chains gs' c aid = Some (ast-c | as-reg-state := s' |)*
using *updated unfolding gs'-def release-lock-chains* **by** *simp*

— Step 7: Extract `reg_state`
then show *?thesis*
unfolding *get-reg-state-def get-asset-state-def* **by** *simp*
qed

15 Valid State Preservation

The central safety property: synchronization preserves global state validity.

If the global state is valid (consistent and unlocked) before sync, and sync succeeds, then the resulting global state is also valid.

This closes the inductive invariant: valid initial state + `valid_state` preservation under `sync` = `valid_state` holds at all reachable states.

The proof decomposes into two parts: (1) `consistent_state` preservation: sync updates all connected chains to the same new state and leaves other assets untouched. (2) `no_locked_without_reason` preservation: sync acquires and then releases the lock within the same operation, so the output has no outstanding locks (assuming no other locks were held).

15.1 Part 1: consistent_state preservation

After sync, any two chains holding asset `aid` agree on its new regulatory state (it is `s'`), and any two chains holding a different asset `aid'` still agree (their states are unchanged from the consistent input).

lemma *sync-preserves-consistent-state*:

assumes *valid*: *valid-state gs*
and *exists*: *asset-exists gs source aid*
and *current*: *get-reg-state gs source aid = Some s*
and *trans*: *reg-transition s action = Some s'*
and *synced*: *sync source action aid gs = Some gs'*
and *fin*: *finite (connected-chains gs aid)*
shows *consistent-state gs'*

```

proof —
  — Unfold sync to extract the structure
  from synced current trans
  obtain gs-locked where
    lock: acquire-lock gs aid = Some gs-locked
    and gs'-def: gs' = release-lock
      (update-all-chains gs-locked aid s' (connected-chains gs aid)) aid
    unfolding sync-def
    by (auto split: option.splits simp: Let-def)

  have locked-chains: gs-chains gs-locked = gs-chains gs
    using acquire-lock-chains[OF lock] by simp

  show ?thesis
  unfolding consistent-state-def
  proof (intro allI impI)
    fix c1 c2 aid' s1-final s2-final
    assume s1-eq: get-reg-state gs' c1 aid' = Some s1-final
    and s2-eq: get-reg-state gs' c2 aid' = Some s2-final

    show s1-final = s2-final
    proof (cases aid' = aid)
      case True
      — Case: the synchronized asset. Two sub-cases per chain.
      show ?thesis
      proof (cases c1 ∈ connected-chains gs aid)
        case c1-in: True
        then have ae1: asset-exists gs c1 aid
          unfolding connected-chains-def by simp
        then obtain ast1 where ast1: gs-chains gs c1 aid = Some ast1
          unfolding asset-exists-def get-asset-state-def by auto
        have ast1-locked: gs-chains gs-locked c1 aid = Some ast1
          using ast1 locked-chains by simp
        have upd1: gs-chains (update-all-chains gs-locked aid s' (connected-chains
gs aid)) c1 aid =
          Some (ast1() as-reg-state := s')
          using update-all-chains-in-targets[OF c1-in ast1-locked] by simp
        have reg1: get-reg-state gs' c1 aid = Some s'
          unfolding gs'-def release-lock-chains get-reg-state-def get-asset-state-def
          using upd1 by simp
        show ?thesis
        proof (cases c2 ∈ connected-chains gs aid)
          case c2-in: True
          then have ae2: asset-exists gs c2 aid
            unfolding connected-chains-def by simp
          then obtain ast2 where ast2: gs-chains gs c2 aid = Some ast2
            unfolding asset-exists-def get-asset-state-def by auto
          have ast2-locked: gs-chains gs-locked c2 aid = Some ast2
            using ast2 locked-chains by simp

```

```

have reg2: get-reg-state  $gs'$   $c2$   $aid = \text{Some } s'$ 
  unfolding  $gs'$ -def release-lock-chains get-reg-state-def get-asset-state-def
  using update-all-chains-in-targets[OF  $c2$ -in  $ast2$ -locked] by simp
  from reg1 reg2  $s1$ -eq  $s2$ -eq  $\text{True}$  show ?thesis by simp
next
  case  $c2$ -out: False
  —  $c2$  is not connected, so asset doesn't exist on  $c2$  for  $aid$ 
  have  $gs$ -chains (update-all-chains  $gs$ -locked  $aid$   $s'$  (connected-chains  $gs$ 
 $aid$ ))  $c2 =$ 
     $gs$ -chains  $gs$ -locked  $c2$ 
    using update-all-chains-outside-targets[OF  $c2$ -out] by simp
  then have  $gs$ -chains  $gs'$   $c2$   $aid = gs$ -chains  $gs$   $c2$   $aid$ 
    unfolding  $gs'$ -def release-lock-chains using locked-chains by simp
  moreover have  $gs$ -chains  $gs$   $c2$   $aid = \text{None}$ 
using  $c2$ -out unfolding connected-chains-def asset-exists-def get-asset-state-def
  by auto
  ultimately have get-reg-state  $gs'$   $c2$   $aid = \text{None}$ 
    unfolding get-reg-state-def get-asset-state-def by simp
  with  $s2$ -eq  $\text{True}$  show ?thesis by simp
qed
next
  case  $c1$ -out: False
  —  $c1$  not connected: asset doesn't exist there
  have  $gs$ -chains (update-all-chains  $gs$ -locked  $aid$   $s'$  (connected-chains  $gs$   $aid$ ))
 $c1 =$ 
     $gs$ -chains  $gs$ -locked  $c1$ 
    using update-all-chains-outside-targets[OF  $c1$ -out] by simp
  then have  $gs$ -chains  $gs'$   $c1$   $aid = gs$ -chains  $gs$   $c1$   $aid$ 
    unfolding  $gs'$ -def release-lock-chains using locked-chains by simp
  moreover have  $gs$ -chains  $gs$   $c1$   $aid = \text{None}$ 
using  $c1$ -out unfolding connected-chains-def asset-exists-def get-asset-state-def
  by auto
  ultimately have get-reg-state  $gs'$   $c1$   $aid = \text{None}$ 
    unfolding get-reg-state-def get-asset-state-def by simp
  with  $s1$ -eq  $\text{True}$  show ?thesis by simp
qed
next
  case  $diff$ : False
  — Case: a different asset. Sync does not touch it.
  have  $other1$ :  $gs$ -chains (update-all-chains  $gs$ -locked  $aid$   $s'$  (connected-chains
 $gs$   $aid$ ))  $c1$   $aid' =$ 
     $gs$ -chains  $gs$ -locked  $c1$   $aid'$ 
    using update-all-chains-other-asset[OF  $diff$ ] by simp
  have  $other2$ :  $gs$ -chains (update-all-chains  $gs$ -locked  $aid$   $s'$  (connected-chains
 $gs$   $aid$ ))  $c2$   $aid' =$ 
     $gs$ -chains  $gs$ -locked  $c2$   $aid'$ 
    using update-all-chains-other-asset[OF  $diff$ ] by simp
  have  $gs$ -chains  $gs'$   $c1$   $aid' = gs$ -chains  $gs$   $c1$   $aid'$ 
    unfolding  $gs'$ -def release-lock-chains using  $other1$  locked-chains by simp

```

```

moreover have gs-chains gs' c2 aid' = gs-chains gs c2 aid'
  unfolding gs'-def release-lock-chains using other2 locked-chains by simp
ultimately have get-reg-state gs' c1 aid' = get-reg-state gs c1 aid'
  and get-reg-state gs' c2 aid' = get-reg-state gs c2 aid'
  unfolding get-reg-state-def get-asset-state-def by simp-all
with s1-eq s2-eq have
  get-reg-state gs c1 aid' = Some s1-final
  get-reg-state gs c2 aid' = Some s2-final
  by simp-all
with valid show ?thesis
  unfolding valid-state-def consistent-state-def by blast
qed
qed
qed

```

15.2 Part 2: no_locked_without_reason preservation

After sync, no assets are locked. The sync operation acquires a lock on the target asset and releases it before returning. For all other assets, the lock state is unchanged (and was False by *valid_state*).

lemma *sync-preserves-no-locks*:

```

assumes valid: valid-state gs
and exists: asset-exists gs source aid
and current: get-reg-state gs source aid = Some s
and trans: reg-transition s action = Some s'
and synced: sync source action aid gs = Some gs'
shows no-locked-without-reason gs'

```

proof —

```

from synced current trans
obtain gs-locked where
  lock: acquire-lock gs aid = Some gs-locked
and gs'-def: gs' = release-lock
  (update-all-chains gs-locked aid s' (connected-chains gs aid)) aid
unfolding sync-def
by (auto split: option.splits simp: Let-def)

```

— Locks after acquire: only aid is locked

```

from lock have locked-locks: gs-locks gs-locked = (gs-locks gs)(aid := True)
unfolding acquire-lock-def is-locked-def
by (auto split: if-splits)

```

— *update_all_chains* does not change locks

```

have upd-locks: gs-locks (update-all-chains gs-locked aid s' (connected-chains gs aid)) =
  gs-locks gs-locked
using update-all-chains-locks by simp

```

— *release_lock* clears the lock on aid

```

have final-locks: gs-locks gs' = (gs-locks gs-locked)(aid := False)

```

```

    unfolding gs'-def release-lock-def using upd-locks by simp

  show ?thesis
  unfolding no-locked-without-reason-def is-locked-def
  proof
    fix aid'
    show  $\neg$  gs-locks gs' aid'
    proof (cases aid' = aid)
      case True
      then show ?thesis using final-locks by simp
    next
      case False
      then have gs-locks gs' aid' = gs-locks gs-locked aid'
        using final-locks by simp
      also have ... = gs-locks gs aid'
        using locked-locks False by simp
      also have ... = False
      using valid unfolding valid-state-def no-locked-without-reason-def is-locked-def
        by simp
      finally show ?thesis by simp
    qed
  qed
qed

```

15.3 The `valid_state` preservation theorem

Combining the two parts: `sync` preserves `valid_state`. This is the inductive invariant that guarantees `valid_state` holds at every reachable global state.

```

theorem valid-state-preservation:
  assumes valid: valid-state gs
    and exists: asset-exists gs source aid
    and current: get-reg-state gs source aid = Some s
    and trans: reg-transition s action = Some s'
    and synced: sync source action aid gs = Some gs'
    and fin: finite (connected-chains gs aid)
  shows valid-state gs'
  unfolding valid-state-def
  using sync-preserves-consistent-state[OF assms]
    sync-preserves-no-locks[OF valid exists current trans synced]
  by auto

```

16 Heterogeneous-Action State Preservation Instance

This section instantiates the `state_preservation` locale with a concrete heterogeneous-action scenario between two chains.

The scenario models the following operational situation. Two chains A

and B share the same regulatory state space (ACTIVE, FROZEN, SEIZED, CONFISCATED, RESTRICTED), but their on-chain action vocabularies differ. Chain A supports the full seven-action set used elsewhere in this theory. Chain B is a chain whose jurisdiction handles de-escalation (UN-FREEZE, UNRESTRICT, RELEASE) exclusively through separate judicial procedures rather than as on-chain actions; correspondingly, Chain B's on-chain action vocabulary is the four-action escalation subset only. The synchronisation map between the two chains is therefore non-trivial in two ways:

- The source action type is `reg_action` and the target action type is a separate datatype `chain_b_action` with four constructors. This exercises the locale's heterogeneous source / target action types ('a vs. 'b).
- The locale parameter `actions\<^sub>s` is instantiated with a strict subset of the full `reg_action` set, namely the four escalation actions. De-escalation actions exist in `reg_action` as a datatype but are out of scope for this synchronisation instance.

The naturality assumption then has to hold only on the escalation subset, which is exactly the design intent of the locale's `actions\<^sub>s` parameter: the user can scope structural preservation to a subset of source actions rather than to the full source action type.

datatype *chain-b-action* = *B-FREEZE* | *B-SEIZE* | *B-CONFISCATE* | *B-RESTRICT*

The escalation subset of `reg_action` that this instance synchronises across to Chain B.

definition *escalation-actions* :: *reg-action* set **where**
escalation-actions = {*FREEZE*, *SEIZE*, *CONFISCATE*, *RESTRICT*}

definition *chain-b-actions* :: *chain-b-action* set **where**
chain-b-actions = {*B-FREEZE*, *B-SEIZE*, *B-CONFISCATE*, *B-RESTRICT*}

Chain B's transition function. Its values mirror Chain A's transition function on the four escalation actions: when Chain A maps (`s`, `escalation action`) to `Some s'`, Chain B does the same; when Chain A rejects, Chain B rejects. Both chains share the regulatory state space, so no state translation is required (the state map is the identity).

fun *chain-b-transition* :: *reg-state* $\Rightarrow$ *chain-b-action* $\Rightarrow$ *reg-state* option **where**
chain-b-transition *s* *B-FREEZE* = *reg-transition* *s* *FREEZE*
| *chain-b-transition* *s* *B-SEIZE* = *reg-transition* *s* *SEIZE*
| *chain-b-transition* *s* *B-CONFISCATE* = *reg-transition* *s* *CONFISCATE*
| *chain-b-transition* *s* *B-RESTRICT* = *reg-transition* *s* *RESTRICT*

The escalation action map: a 1-to-1 correspondence between Chain A's four escalation actions and Chain B's four constructors.

```

fun escalation-action-map :: reg-action  $\Rightarrow$  chain-b-action where
  escalation-action-map FREEZE    = B-FREEZE
| escalation-action-map SEIZE     = B-SEIZE
| escalation-action-map CONFISCATE = B-CONFISCATE
| escalation-action-map RESTRICT  = B-RESTRICT
| escalation-action-map UNFREEZE  = B-FREEZE
| escalation-action-map UNRESTRICT = B-FREEZE
| escalation-action-map RELEASE   = B-FREEZE
  — The last three clauses are unused by the locale instance, since actions\<^sub>s
  = escalation_actions excludes the de-escalation actions. They are present only
  to make the function total on reg_action.

```

Chain B is also a state machine over the same regulatory state space.

```

lemma chain-b-transition-closed:
  chain-b-transition s a = Some s'  $\implies$  s'  $\in$  reg-states
proof —
  assume chain-b-transition s a = Some s'
  then have  $\exists$  a'. reg-transition s a' = Some s'
    by (cases a) auto
  then show s'  $\in$  reg-states using reg-transition-closed by auto
qed

```

```

lemma chain-b-terminal:
  s  $\in$  reg-terminal  $\implies$  chain-b-transition s a = None
  unfolding reg-terminal-def
  by (cases a) (auto simp: confiscated-terminal)

```

```

lemma chain-b-state-machine:
  state-machine reg-states chain-b-actions chain-b-transition reg-terminal
proof —
  have f1: finite reg-states unfolding reg-states-def by auto
  have f2: finite chain-b-actions unfolding chain-b-actions-def by auto
  have f3: reg-terminal  $\subseteq$  reg-states
    unfolding reg-terminal-def reg-states-def by auto
  have f4:  $\bigwedge$  s a. s  $\in$  reg-terminal  $\implies$  a  $\in$  chain-b-actions  $\implies$  chain-b-transition
    s a = None
    using chain-b-terminal by auto
  have f5:  $\bigwedge$  s a s'. s  $\in$  reg-states  $\implies$  a  $\in$  chain-b-actions  $\implies$ 
    chain-b-transition s a = Some s'  $\implies$  s'  $\in$  reg-states
    using chain-b-transition-closed by auto
  have f6:  $\bigwedge$  s a. (s :: reg-state)  $\notin$  reg-states  $\implies$  chain-b-transition s a = None
    using reg-states-UNIV by auto
  show ?thesis
    by (intro state-machine.intro) (use f1 f2 f3 f4 f5 f6 in auto)
qed

```

The naturality conditions of `state_preservation` hold for the escalation subset. Both directions (`Some` and `None` branches) follow by case analysis on the escalation action: by construction, Chain B's transition function on each `B_X` constructor mirrors Chain A's transition function on the corresponding

X action.

lemma *escalation-naturality-some*:

assumes $a \in \text{escalation-actions}$

and $\text{reg-transition } s \ a = \text{Some } s'$

shows $\text{chain-b-transition } s \ (\text{escalation-action-map } a) = \text{Some } s'$

proof –

from $\text{assms}(1)$ **have** $a \in \{\text{FREEZE}, \text{SEIZE}, \text{CONFISCATE}, \text{RESTRICT}\}$

unfolding $\text{escalation-actions-def}$ **by** simp

then consider $a = \text{FREEZE} \mid a = \text{SEIZE} \mid a = \text{CONFISCATE} \mid a = \text{RESTRICT}$

by auto

then show $?thesis$ **using** $\text{assms}(2)$ **by** $(\text{cases}) \text{ auto}$

qed

lemma *escalation-naturality-none*:

assumes $a \in \text{escalation-actions}$

and $\text{reg-transition } s \ a = \text{None}$

shows $\text{chain-b-transition } s \ (\text{escalation-action-map } a) = \text{None}$

proof –

from $\text{assms}(1)$ **have** $a \in \{\text{FREEZE}, \text{SEIZE}, \text{CONFISCATE}, \text{RESTRICT}\}$

unfolding $\text{escalation-actions-def}$ **by** simp

then consider $a = \text{FREEZE} \mid a = \text{SEIZE} \mid a = \text{CONFISCATE} \mid a = \text{RESTRICT}$

by auto

then show $?thesis$ **using** $\text{assms}(2)$ **by** $(\text{cases}) \text{ auto}$

qed

The escalation instance: `state_preservation` with actions\<^sub>s the escalation subset of `reg_action`, target action type `chain_b_action`, and identity state map (both chains share the regulatory state space).

Source and target state-machine bundles for the heterogeneous-action instance. The target side reuses the existing `chain_b_state_machine` lemma; the source side is on the escalation subset of `reg_actions` rather than the full action set, so we discharge it inline.

lemma *escalation-source-state-machine*:

state-machine reg-states escalation-actions reg-transition reg-terminal

proof *unfold-locales*

show *finite reg-states* **unfolding** reg-states-def **by** auto

next

show *finite escalation-actions* **unfolding** $\text{escalation-actions-def}$ **by** auto

next

show $\text{reg-terminal} \subseteq \text{reg-states}$ **unfolding** reg-terminal-def reg-states-def **by** auto

next

fix $s \ a$

assume $s \in \text{reg-terminal}$ $a \in \text{escalation-actions}$

then show $\text{reg-transition } s \ a = \text{None}$

unfolding reg-terminal-def **by** $(\text{auto simp: confiscated-terminal})$

next

```

fix  $s\ a\ s'$ 
assume  $s \in \text{reg-states}\ a \in \text{escalation-actions}\ \text{reg-transition}\ s\ a = \text{Some}\ s'$ 
then show  $s' \in \text{reg-states}$  using  $\text{reg-transition-closed}$  by  $\text{auto}$ 
next
fix  $s :: \text{reg-state}\ \text{and}\ a :: \text{reg-action}$ 
assume  $s \notin \text{reg-states}$ 
then show  $\text{reg-transition}\ s\ a = \text{None}$  using  $\text{reg-states-UNIV}$  by  $\text{auto}$ 
qed

```

Heterogeneous-action instance: the source action set is the `escalation_actions` subset of `reg_action`, the target action set is `chain_b_actions`, and the state map is the identity. Both source and target state machines share the regulatory state space, so two of the state-machine obligations (`finite reg_states` and `reg_terminal \subseteq reg_states`) collapse between source and target; the proof structure below reflects this by issuing each `show` exactly once for the merged obligation.

interpretation *escalation-preservation*:

```

state-preservation
   $\text{reg-states}\ \text{escalation-actions}\ \text{reg-transition}\ \text{reg-terminal}$ 
   $\text{reg-states}\ \text{chain-b-actions}\ \text{chain-b-transition}\ \text{reg-terminal}$ 
   $\text{id} :: \text{reg-state} \Rightarrow \text{reg-state}\ \text{escalation-action-map}$ 
proof unfold-locales
  show finite reg-states unfolding  $\text{reg-states-def}$  by  $\text{auto}$ 
next
  show finite escalation-actions unfolding  $\text{escalation-actions-def}$  by  $\text{auto}$ 
next
  show  $\text{reg-terminal} \subseteq \text{reg-states}$  unfolding  $\text{reg-terminal-def}\ \text{reg-states-def}$  by  $\text{auto}$ 
next
  fix  $s\ a$ 
  assume  $s \in \text{reg-terminal}\ a \in \text{escalation-actions}$ 
  then show  $\text{reg-transition}\ s\ a = \text{None}$ 
    unfolding  $\text{reg-terminal-def}$  by ( $\text{auto simp: confiscated-terminal}$ )
next
  fix  $s\ a\ s'$ 
  assume  $s \in \text{reg-states}\ a \in \text{escalation-actions}\ \text{reg-transition}\ s\ a = \text{Some}\ s'$ 
  then show  $s' \in \text{reg-states}$  using  $\text{reg-transition-closed}$  by  $\text{auto}$ 
next
  fix  $s :: \text{reg-state}\ \text{and}\ a :: \text{reg-action}$ 
  assume  $s \notin \text{reg-states}$ 
  then show  $\text{reg-transition}\ s\ a = \text{None}$  using  $\text{reg-states-UNIV}$  by  $\text{auto}$ 
next
  show finite chain-b-actions unfolding  $\text{chain-b-actions-def}$  by  $\text{auto}$ 
next
  fix  $s\ a$ 
  assume  $s \in \text{reg-terminal}\ a \in \text{chain-b-actions}$ 
  then show  $\text{chain-b-transition}\ s\ a = \text{None}$  using  $\text{chain-b-terminal}$  by  $\text{auto}$ 
next
  fix  $s\ a\ s'$ 
  assume  $s \in \text{reg-states}\ a \in \text{chain-b-actions}\ \text{chain-b-transition}\ s\ a = \text{Some}\ s'$ 

```

```

    then show  $s' \in \text{reg-states}$  using chain-b-transition-closed by auto
next
  fix  $s :: \text{reg-state}$  and  $a :: \text{chain-b-action}$ 
  assume  $s \notin \text{reg-states}$ 
  then show chain-b-transition  $s a = \text{None}$  using reg-states-UNIV by auto
next
  fix  $s$ 
  assume  $s \in \text{reg-states}$ 
  then show  $\text{id } s \in \text{reg-states}$  by simp
next
  fix  $a$ 
  assume  $a \in \text{escalation-actions}$ 
  then show escalation-action-map  $a \in \text{chain-b-actions}$ 
    unfolding escalation-actions-def chain-b-actions-def by auto
next
  fix  $s$ 
  assume  $s \in \text{reg-terminal}$ 
  then show  $\text{id } s \in \text{reg-terminal}$  by simp
next
  fix  $s a s'$ 
  assume  $s \in \text{reg-states } a \in \text{escalation-actions } \text{reg-transition } s a = \text{Some } s'$ 
  then show chain-b-transition  $(\text{id } s) (\text{escalation-action-map } a) = \text{Some } (\text{id } s')$ 
    using escalation-naturality-some by simp
next
  fix  $s a$ 
  assume  $s \in \text{reg-states } a \in \text{escalation-actions } \text{reg-transition } s a = \text{None}$ 
  then show chain-b-transition  $(\text{id } s) (\text{escalation-action-map } a) = \text{None}$ 
    using escalation-naturality-none by simp
qed

```

As a corollary of the locale interpretation, sequential preservation (the locale's main theorem) applies to escalation action sequences: any valid sequence of escalation actions on Chain A produces, when mapped through `escalation_action_map`, a valid sequence on Chain B ending in the same regulatory state.

corollary *escalation-sequential-preservation*:

```

  assumes  $s \in \text{reg-states}$ 
    and  $\forall a \in \text{set } as. a \in \text{escalation-actions}$ 
    and escalation-preservation.source.apply-actions  $s as = \text{Some } s'$ 
  shows escalation-preservation.target.apply-actions  $s (\text{map } \text{escalation-action-map } as)$ 
    =  $\text{Some } s'$ 
  using assms escalation-preservation.sequential-preservation [where  $as = as$ ]
  by simp

```

17 Layer-Crossing Symmetric State Preservation Instance

This section instantiates the `symmetric_state_preservation` locale with a concrete bidirectional binding between two representations of the same regulatory state.

The on-chain representation is the five-element `reg_state` enum used throughout this theory. The off-chain representation is a structured permission record (`daml_perm`) modelling a DAML party-permission ledger entry: a status tag plus auxiliary fields that carry the party / scope metadata associated with a non-trivial regulatory status (the seizing party for `SEIZED`, the restriction scope for `RESTRICTED`). A type-level invariant (`valid_daml_perm`) carves out the bijection domain by tying the auxiliary fields to the status tag.

The action vocabularies on the two layers coincide: both layers process the seven actions of `reg_action`, and the action map is the identity. All of the non-trivial content of the symmetric instance therefore lives in the layer-crossing state mapping. The roundtrip assumptions of `symmetric_state_preservation` become a pair of bijection conditions between `reg_state` and the valid `daml_perm` records.

```
datatype daml-status-tag =
  D-Active | D-Frozen | D-Seized | D-Confiscated | D-Restricted
```

```
record daml-perm =
  status-tag      :: daml-status-tag
  seized-by      :: nat option
  restriction-scope :: nat option
```

```
definition default-party-id :: nat where
  default-party-id = 0
```

```
definition default-scope-id :: nat where
  default-scope-id = 0
```

The well-formedness invariant: the auxiliary fields are non-`None` exactly on the status tags that semantically require them.

```
definition valid-daml-perm :: daml-perm  $\Rightarrow$  bool where
  valid-daml-perm p  $\longleftrightarrow$ 
    (status-tag p = D-Seized  $\longleftrightarrow$  seized-by p  $\neq$  None)  $\wedge$ 
    (status-tag p = D-Restricted  $\longleftrightarrow$  restriction-scope p  $\neq$  None)
```

The two state maps between the two representations.

```
fun reg-to-daml :: reg-state  $\Rightarrow$  daml-perm where
  reg-to-daml ACTIVE      =  $\langle$  status-tag = D-Active,
                               seized-by = None,
                               restriction-scope = None  $\rangle$ 
```

```

| reg-to-daml FROZEN      = (| status-tag = D-Frozen,
                             seized-by = None,
                             restriction-scope = None |)
| reg-to-daml SEIZED      = (| status-tag = D-Seized,
                             seized-by = Some default-party-id,
                             restriction-scope = None |)
| reg-to-daml CONFISCATED = (| status-tag = D-Confiscated,
                             seized-by = None,
                             restriction-scope = None |)
| reg-to-daml RESTRICTED = (| status-tag = D-Restricted,
                             seized-by = None,
                             restriction-scope = Some default-scope-id |)

```

```

fun daml-to-reg :: daml-perm ⇒ reg-state where
  daml-to-reg p = (case status-tag p of
    D-Active      ⇒ ACTIVE
  | D-Frozen      ⇒ FROZEN
  | D-Seized      ⇒ SEIZED
  | D-Confiscated ⇒ CONFISCATED
  | D-Restricted  ⇒ RESTRICTED)

```

The DAML target state space: the image of `reg_to_daml`.

```

definition daml-states :: daml-perm set where
  daml-states = range reg-to-daml

```

```

definition daml-terminal :: daml-perm set where
  daml-terminal = {reg-to-daml CONFISCATED}

```

The DAML transition function: lifts `reg_transition` through the representation map. By construction, `daml_transition` respects the `reg_to_daml` map.

```

definition daml-transition :: daml-perm ⇒ reg-action ⇒ daml-perm option where
  daml-transition p a =
    (if p ∈ daml-states then
      (case reg-transition (daml-to-reg p) a of
        None      ⇒ None
      | Some s' ⇒ Some (reg-to-daml s'))
    else None)

```

Roundtrip lemmas establishing the layer-crossing bijection.

```

lemma daml-to-reg-to-daml-id:
  daml-to-reg (reg-to-daml s) = s
by (cases s) auto

```

```

lemma reg-to-daml-to-reg-on-image:
  assumes p ∈ daml-states
  shows reg-to-daml (daml-to-reg p) = p
proof –
  from assms obtain s where p = reg-to-daml s

```

```

    unfolding daml-states-def by auto
  then show ?thesis using daml-to-reg-to-daml-id by simp
qed

```

```

lemma reg-to-daml-valid:
  valid-daml-perm (reg-to-daml s)
  unfolding valid-daml-perm-def by (cases s) auto

```

```

lemma reg-to-daml-injective:
  reg-to-daml s1 = reg-to-daml s2  $\implies$  s1 = s2
  using daml-to-reg-to-daml-id by metis

```

The DAML side is a state machine on the image of `reg_to_daml`.

```

lemma daml-states-finite: finite daml-states
proof -
  have fin-reg: finite reg-states unfolding reg-states-def by auto
  hence fin-img: finite (reg-to-daml ` reg-states) by (rule finite-imageI)
  have eq: daml-states = reg-to-daml ` reg-states
    unfolding daml-states-def using reg-states-UNIV by simp
  from fin-img eq show ?thesis by simp
qed

```

```

lemma daml-terminal-subset: daml-terminal  $\subseteq$  daml-states
  unfolding daml-terminal-def daml-states-def by blast

```

```

lemma daml-terminal-absorbing:
  assumes p  $\in$  daml-terminal and a  $\in$  reg-actions
  shows daml-transition p a = None
proof -
  from assms(1) have p = reg-to-daml CONFISCATED
    unfolding daml-terminal-def by simp
  then have daml-to-reg p = CONFISCATED using daml-to-reg-to-daml-id by
simp
  moreover have p  $\in$  daml-states
    using assms(1) daml-terminal-subset by auto
  ultimately show ?thesis
    unfolding daml-transition-def using confiscated-terminal by simp
qed

```

```

lemma daml-transition-closed:
  assumes p  $\in$  daml-states and a  $\in$  reg-actions
  and daml-transition p a = Some p'
  shows p'  $\in$  daml-states
proof -
  from assms(1,3) obtain s' where
    s'-eq: reg-transition (daml-to-reg p) a = Some s'
    and p'-def: p' = reg-to-daml s'
    unfolding daml-transition-def
    by (auto split: option.splits if-splits)

```



```

    then show ?thesis
      unfolding daml-states-def by auto
qed

```

```

lemma daml-transition-outside-states:
   $p \notin \text{daml-states} \implies \text{daml-transition } p \ a = \text{None}$ 
  unfolding daml-transition-def by simp

```

```

lemma daml-state-machine:
  state-machine daml-states reg-actions daml-transition daml-terminal
proof
  show finite daml-states by (rule daml-states-finite)
next
  show finite reg-actions unfolding reg-actions-def by auto
next
  show  $\text{daml-terminal} \subseteq \text{daml-states}$  by (rule daml-terminal-subset)
next
  fix s a
  assume  $s \in \text{daml-terminal} \ a \in \text{reg-actions}$ 
  then show  $\text{daml-transition } s \ a = \text{None}$  by (rule daml-terminal-absorbing)
next
  fix s a s'
  assume  $s \in \text{daml-states} \ a \in \text{reg-actions} \ \text{daml-transition } s \ a = \text{Some } s'$ 
  then show  $s' \in \text{daml-states}$  by (rule daml-transition-closed)
next
  fix s :: daml-perm and a :: reg-action
  assume  $s \notin \text{daml-states}$ 
  then show  $\text{daml-transition } s \ a = \text{None}$  by (rule daml-transition-outside-states)
qed

```

Forward naturality: `reg_to_daml` commutes with transitions.

```

lemma reg-to-daml-naturality-some:
  assumes  $s \in \text{reg-states}$  and  $a \in \text{reg-actions}$ 
  and  $\text{reg-transition } s \ a = \text{Some } s'$ 
  shows  $\text{daml-transition } (\text{reg-to-daml } s) \ a = \text{Some } (\text{reg-to-daml } s')$ 
proof -
  have in-states:  $\text{reg-to-daml } s \in \text{daml-states}$ 
  unfolding daml-states-def by auto
  have  $\text{daml-to-reg } (\text{reg-to-daml } s) = s$  using daml-to-reg-to-daml-id by simp
  with assms(3) have
     $\text{reg-transition } (\text{daml-to-reg } (\text{reg-to-daml } s)) \ a = \text{Some } s'$  by simp
  with in-states show ?thesis
    unfolding daml-transition-def by simp
qed

```

```

lemma reg-to-daml-naturality-none:
  assumes  $s \in \text{reg-states}$  and  $a \in \text{reg-actions}$ 
  and  $\text{reg-transition } s \ a = \text{None}$ 
  shows  $\text{daml-transition } (\text{reg-to-daml } s) \ a = \text{None}$ 

```

proof –
 have *in-states*: $\text{reg-to-daml } s \in \text{daml-states}$
 unfolding *daml-states-def* by *auto*
 have $\text{daml-to-reg } (\text{reg-to-daml } s) = s$ using *daml-to-reg-to-daml-id* by *simp*
 with *assms*(3) have
 $\text{reg-transition } (\text{daml-to-reg } (\text{reg-to-daml } s)) \ a = \text{None}$ by *simp*
 with *in-states* show *?thesis*
 unfolding *daml-transition-def* by *simp*
qed

Backward naturality: *daml_to_reg* on the image of *reg_to_daml* commutes with transitions in the opposite direction.

lemma *daml-to-reg-naturality-some*:
 assumes $p \in \text{daml-states}$ and $a \in \text{reg-actions}$
 and $\text{daml-transition } p \ a = \text{Some } p'$
 shows $\text{reg-transition } (\text{daml-to-reg } p) \ a = \text{Some } (\text{daml-to-reg } p')$
proof –
 from *assms*(1,3) obtain s' where
 $s'\text{-eq}$: $\text{reg-transition } (\text{daml-to-reg } p) \ a = \text{Some } s'$
 and $p'\text{-def}$: $p' = \text{reg-to-daml } s'$
 unfolding *daml-transition-def*
 by (*auto split: option.splits if-splits*)
 from $p'\text{-def}$ have $\text{daml-to-reg } p' = s'$ using *daml-to-reg-to-daml-id* by *simp*
 with $s'\text{-eq}$ show *?thesis* by *simp*
qed

lemma *daml-to-reg-naturality-none*:
 assumes $p \in \text{daml-states}$ and $a \in \text{reg-actions}$
 and $\text{daml-transition } p \ a = \text{None}$
 shows $\text{reg-transition } (\text{daml-to-reg } p) \ a = \text{None}$
proof (*rule ccontr*)
 assume $\text{reg-transition } (\text{daml-to-reg } p) \ a \neq \text{None}$
 then obtain s' where $\text{reg-transition } (\text{daml-to-reg } p) \ a = \text{Some } s'$ by *auto*
 with *assms*(1) have $\text{daml-transition } p \ a = \text{Some } (\text{reg-to-daml } s')$
 unfolding *daml-transition-def* by *simp*
 with *assms*(3) show *False* by *simp*
qed

Forward state preservation: *reg_to_daml* as a structure-preserving map.

Forward state preservation. The source state machine is on (*reg_states*, *reg_actions*, *reg_transition*, *reg_terminal*), which exactly matches the *reg_sm* interpretation; consequently *unfold_locales* auto-discharges all six source state-machine obligations. The target side is the DAML structured-permission record, for which only individual lemmas (not a registered interpretation) are available, so all six target state-machine obligations remain pending alongside the five preservation-specific axioms eleven obligations in total. The asymmetry with *backward_layer_preservation* below (which has only the five preservation axioms remaining) reflects the order

in which the locale extension presents the two state-machine sub-locales.

interpretation *forward-layer-preservation*:

state-preservation

reg-states reg-actions reg-transition reg-terminal

daml-states reg-actions daml-transition daml-terminal

reg-to-daml id

proof *unfold-locales*

show *finite daml-states* **by** (*rule daml-states-finite*)

next

show *finite reg-actions* **unfolding** *reg-actions-def* **by** *auto*

next

show *daml-terminal* $\subseteq$ *daml-states* **by** (*rule daml-terminal-subset*)

next

fix *s a*

assume *s* $\in$ *daml-terminal* *a* $\in$ *reg-actions*

then show *daml-transition s a* = *None* **by** (*rule daml-terminal-absorbing*)

next

fix *s a s'*

assume *s* $\in$ *daml-states* *a* $\in$ *reg-actions* *daml-transition s a* = *Some s'*

then show *s'* $\in$ *daml-states* **by** (*rule daml-transition-closed*)

next

fix *s :: daml-perm* **and** *a :: reg-action*

assume *s* $\notin$ *daml-states*

then show *daml-transition s a* = *None* **by** (*rule daml-transition-outside-states*)

next

fix *s*

assume *s* $\in$ *reg-states*

then show *reg-to-daml s* $\in$ *daml-states* **unfolding** *daml-states-def* **by** *auto*

next

fix *a*

assume *a* $\in$ *reg-actions*

then show *id a* $\in$ *reg-actions* **by** *simp*

next

fix *s*

assume *s* $\in$ *reg-terminal*

then show *reg-to-daml s* $\in$ *daml-terminal*

unfolding *reg-terminal-def daml-terminal-def* **by** *simp*

next

fix *s a s'*

assume *s* $\in$ *reg-states* *a* $\in$ *reg-actions* *reg-transition s a* = *Some s'*

then show *daml-transition (reg-to-daml s) (id a)* = *Some (reg-to-daml s')*

using *reg-to-daml-naturality-some* **by** *simp*

next

fix *s a*

assume *s* $\in$ *reg-states* *a* $\in$ *reg-actions* *reg-transition s a* = *None*

then show *daml-transition (reg-to-daml s) (id a)* = *None*

using *reg-to-daml-naturality-none* **by** *simp*

qed

Backward state preservation: `daml_to_reg` as a structure-preserving map on the image of `reg_to_daml`.

Backward state preservation. The source side is DAML and the target side matches `reg_sm`. `unfold_locales` auto-discharges all twelve state-machine obligations (six target via `reg_sm`, six source via the bundled `daml_state_machine` lemma combined with the individual `daml_*` lemmas), so only the five preservation axioms remain pending.

interpretation *backward-layer-preservation:*

```

  state-preservation
    daml-states reg-actions daml-transition daml-terminal
    reg-states reg-actions reg-transition reg-terminal
    daml-to-reg id
proof unfold-locales
  fix p
    assume p ∈ daml-states
    then show daml-to-reg p ∈ reg-states using reg-states-UNIV by auto
next
  fix a
    assume a ∈ reg-actions
    then show id a ∈ reg-actions by simp
next
  fix p
    assume p ∈ daml-terminal
    then have p = reg-to-daml CONFISCATED unfolding daml-terminal-def by
      simp
    then have daml-to-reg p = CONFISCATED using daml-to-reg-to-daml-id by
      simp
    then show daml-to-reg p ∈ reg-terminal unfolding reg-terminal-def by simp
next
  fix p a p'
    assume p ∈ daml-states a ∈ reg-actions daml-transition p a = Some p'
    then show reg-transition (daml-to-reg p) (id a) = Some (daml-to-reg p')
      using daml-to-reg-naturality-some by simp
next
  fix p a
    assume p ∈ daml-states a ∈ reg-actions daml-transition p a = None
    then show reg-transition (daml-to-reg p) (id a) = None
      using daml-to-reg-naturality-none by simp
qed

```

The symmetric instance: forward and backward state preservation composed with roundtrip guarantees. The non-trivial content is the bijection between `reg_state` and the image of `reg_to_daml` in `daml_perm`; the action map is the identity because both layers share the same regulatory action vocabulary.

interpretation *onchain-daml-bridge:*

symmetric-state-preservation

```

    reg-states reg-actions reg-transition reg-terminal
    daml-states reg-actions daml-transition daml-terminal
    reg-to-daml id
    daml-to-reg id
proof unfold-locales
  fix s
  assume s ∈ reg-states
  then show daml-to-reg (reg-to-daml s) = s using daml-to-reg-to-daml-id by simp
next
  fix p
  assume p ∈ daml-states
  then show reg-to-daml (daml-to-reg p) = p using reg-to-daml-to-reg-on-image
by simp
next
  fix a
  assume a ∈ reg-actions
  then show id (id a) = a by simp
qed

```

As a corollary of the symmetric interpretation, `reg_to_daml` is injective on `reg_states`: distinct on-chain states map to distinct off-chain DAML records, so the layer-crossing representation incurs no information loss.

corollary *reg-to-daml-injective-on-states*:

```

  [ s1 ∈ reg-states; s2 ∈ reg-states; reg-to-daml s1 = reg-to-daml s2 ]
  ⇒ s1 = s2
using onchain-daml-bridge.state-map-injective .

```

18 Multi-Domain Locale Instantiation

We now instantiate the `multi_domain_preservation` locale from `State_Preservation.thy` with the regulatory state machine.

The instantiation is parametric: given ANY finite set of domains and ANY initial `domain_state` function satisfying the consistency precondition, the locale is satisfied. This proves that the abstract framework applies to the concrete regulatory model without fixing a specific topology.

The key bridge: we construct a `domain_state` function from our `global_state`, and show that `valid_state` implies the consistency precondition required by `multi_domain_preservation`.

Bridge from our `global_state` model to the `multi_domain_preservation` locale's `domain_state` signature.

```

multi_domain_preservation expects: domain_state :: 'd => 'id =>
's option

```

We instantiate with: `'d = chain_id`, `'id = asset_id`, `'s = reg_state`

The bridge function extracts `reg_state` from our richer `asset_state`.

definition *reg-domain-state* :: *global-state* ⇒ *chain-id* ⇒ *asset-id* ⇒ *reg-state option* **where**

reg-domain-state gs cid aid = get-reg-state gs cid aid

The parametric instantiation theorem: for any finite set of `chain_ids` and any valid `global_state`, we can interpret the `multi_domain_preservation` locale with the regulatory state machine.

The `state_machine` predicate as a standalone fact. This is needed for instantiating `multi_domain_preservation`, which takes `state_machine` as an opaque predicate assumption.

lemma *reg-state-machine-pred*:

state-machine reg-states reg-actions reg-transition reg-terminal

proof –

have *f1*: *finite reg-states* **unfolding** *reg-states-def* **by** *auto*

have *f2*: *finite reg-actions* **unfolding** *reg-actions-def* **by** *auto*

have *f3*: *reg-terminal* $\subseteq$ *reg-states*

unfolding *reg-terminal-def reg-states-def* **by** *auto*

have *f4*: $\bigwedge s a. s \in \text{reg-terminal} \implies a \in \text{reg-actions} \implies \text{reg-transition } s a = \text{None}$

unfolding *reg-terminal-def* **by** (*auto simp: confiscated-terminal*)

have *f5*: $\bigwedge s a s'. s \in \text{reg-states} \implies a \in \text{reg-actions} \implies \text{reg-transition } s a = \text{Some } s' \implies s' \in \text{reg-states}$

using *reg-transition-closed* **by** *auto*

have *f6*: $\bigwedge s a. (s :: \text{reg-state}) \notin \text{reg-states} \implies \text{reg-transition } s a = \text{None}$

using *reg-states-UNIV* **by** *auto*

show *?thesis*

by (*intro state-machine.intro*) (*use f1 f2 f3 f4 f5 f6 in auto*)

qed

theorem *reg-multi-domain-instantiation*:

assumes *fin-doms*: *finite (doms :: chain-id set)*

and *valid*: *valid-state gs*

shows *multi-domain-preservation doms reg-states reg-actions reg-transition reg-terminal (reg-domain-state gs)*

proof (*intro multi-domain-preservation.intro*)

show *finite doms* **using** *fin-doms* **by** *simp*

next

show *state-machine reg-states reg-actions reg-transition reg-terminal*

by (*rule reg-state-machine-pred*)

next

fix *d1 d2 :: chain-id* **and** *aid :: asset-id* **and** *s1 s2 :: reg-state*

assume *d1* $\in$ *doms* *d2* $\in$ *doms*

and *reg-domain-state gs d1 aid = Some s1*

and *reg-domain-state gs d2 aid = Some s2*

then have *get-reg-state gs d1 aid = Some s1*

and *get-reg-state gs d2 aid = Some s2*

unfolding *reg-domain-state-def* **by** *simp-all*

with *valid* **show** *s1 = s2*

unfolding *valid-state-def consistent-state-def* **by** *blast*

qed

Corollary: the generic `cross_domain_consistency` theorem from `State_Preservation.thy` applies to regulatory synchronization.

This connects the abstract theory to the concrete model: the generic theorem guarantees that after `sync_all` (the abstract synchronization), all connected domains agree. Combined with our concrete `regulatory_homomorphism` theorem, we have both the abstract and concrete guarantees.

The instantiation above (`reg_multi_domain_instantiation`) enables using all theorems proven in `multi_domain_preservation` in particular `cross_domain_consistency` and `sync_isolation` for the regulatory model without reproving them. Any consumer of this theory can invoke:

```
multi_domain_preservation.cross_domain_consistency [OF reg_multi_domain_instantiation
  fin_doms valid_gs]]
```

to obtain the concrete guarantee for their domain set and global state.

Summary of what this theory establishes:

1. The regulatory state machine (`reg_state`, `reg_action`, `reg_transition`) satisfies the `state_machine` locale from `State_Preservation.thy`.

2. `CONFISCATED` is terminal (I1), `CONFISCATE` is universally reachable (I2), and the transition function is deterministic (I3).

3. `SEIZED` to `FROZEN` and `FROZEN` to `RESTRICTED` direct transitions are excluded by design, with legal and complexity-reduction justifications.

4. The synchronization protocol (lock, then update, then unlock) preserves cross-chain consistency for the same asset while isolating other assets.

5. Preemptive locking prevents concurrent regulatory actions on the same asset (NOT double-spend prevention that is handled at a different layer).

6. The `regulatory_homomorphism` theorem establishes that after sync, all connected chains agree on the new regulatory state.

7. The `valid_state_preservation` theorem establishes that sync preserves the global validity invariant: `consistent_state` AND no outstanding locks. This closes the inductive invariant.

8. The heterogeneous-action instance (`escalation_preservation`) instantiates `state_preservation` with the four-action escalation subset of `reg_action` on the source side, the dedicated four-constructor datatype `chain_b_action` on the target side, and the identity state map. This exercises both the locale's `actions\<^sub>s` subset parameter and the heterogeneous source / target action types.

9. The layer-crossing instance (`onchain_daml_bridge`) instantiates `symmetric_state_preservation` with the on-chain enum representation and a structured DAML permission record (`daml_perm`) carrying auxiliary fields. The action vocabularies coincide so the action map is the identity; the non-trivial content is the bijection between the enum and the valid records, with a type-level invariant (`valid_daml_perm`) carving out the bijection domain.

10. The `multi_domain_preservation` locale is instantiated parametrically: for any finite domain set and valid global state, the abstract framework from `State_Preservation.thy` applies to the regulatory model.

Open work for subsequent entries: - Eventual consistency under the partially synchronous network model (with message delays, redelivery, and out-of-order arrival).

end

```
theory Priority-Resolution
  imports Main
begin
```

19 Priority-Based Deterministic Selection

A system where messages carry priorities from a linear order. Given a finite non-empty set of messages with injective priorities, there exists a unique highest-priority message, and selection is deterministic.

The injectivity assumption models tiebreaking: in practice, a composite key (e.g., authority level, timestamp, severity, node ID) ensures no two distinct messages share the same priority.

```
locale priority-system =
  fixes priority :: 'm  $\Rightarrow$  'k::linorder
  assumes priority-injective:
     $\llbracket \text{priority } m1 = \text{priority } m2 \rrbracket \implies m1 = m2$ 
begin
```

In a finite non-empty set with injective priorities, there exists a unique element with the maximum priority.

```
lemma highest-priority-exists:
  assumes finite S and S  $\neq \{\}$ 
  shows  $\exists! m. m \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } m)$ 
proof -
  have fin-img: finite (priority ` S) using assms(1) by simp
  have ne-img: priority ` S  $\neq \{\}$  using assms(2) by simp
  obtain m where m-in: m  $\in$  S and m-pri: priority m = Max (priority ` S)
    using Max-in[OF fin-img ne-img] by auto
  have m-max:  $\forall m' \in S. \text{priority } m' \leq \text{priority } m$ 
  proof
    fix m' assume m'  $\in$  S
    hence priority m'  $\in$  priority ` S by simp
    hence priority m'  $\leq$  Max (priority ` S) using Max-ge[OF fin-img] by simp
    thus priority m'  $\leq$  priority m using m-pri by simp
  qed
show ?thesis
proof (rule ex1I[of - m])
```



```

  show  $m \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } m)$ 
    using m-in m-max by auto
next
  fix m2 assume  $m2 \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } m2)$ 
  then have m2-in:  $m2 \in S$  and m2-max:  $\forall m' \in S. \text{priority } m' \leq \text{priority } m2$ 
by auto
  from m-max m2-in have  $\text{priority } m2 \leq \text{priority } m$  by auto
  from m2-max m-in have  $\text{priority } m \leq \text{priority } m2$  by auto
  then have  $\text{priority } m = \text{priority } m2$  using  $\langle \text{priority } m2 \leq \text{priority } m \rangle$  by auto
  then show  $m2 = m$  using priority-injective by auto
qed
qed

```

definition *select-highest* :: '*m* set $\Rightarrow$ '*m* option **where**

```

select-highest S =
  (if  $S = \{\}$  then None
   else Some (THE m.  $m \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } m)$ ))

```

theorem *select-highest-deterministic*:

```

  assumes fin: finite S and ne:  $S \neq \{\}$ 
  shows  $\exists! m. \text{select-highest } S = \text{Some } m$ 
proof -
  from ne obtain v where hv:  $\text{select-highest } S = \text{Some } v$ 
  unfolding select-highest-def by simp
  show ?thesis
  proof (rule ex1I[of - v])
    show  $\text{select-highest } S = \text{Some } v$  by (rule hv)
  next
    fix w assume  $\text{select-highest } S = \text{Some } w$ 
    with hv show  $w = v$  by simp
  qed
qed

```

theorem *select-highest-in-set*:

```

  assumes fin: finite S and sel:  $\text{select-highest } S = \text{Some } m$ 
  shows  $m \in S$ 
proof -
  from sel have ne:  $S \neq \{\}$ 
  unfolding select-highest-def by (simp split: if-splits)
  from highest-priority-exists[OF fin ne] have uniq:
     $\exists! x. x \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } x)$  .
  from theI'[OF uniq] have the-in:
     $(\text{THE } x. x \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } x)) \in S$  by auto
  from sel ne have  $m = (\text{THE } x. x \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } x))$ 
  unfolding select-highest-def by simp
  with the-in show ?thesis by simp
qed

```

theorem *select-highest-is-max*:

assumes *fin*: finite *S* **and** *ne*: $S \neq \{\}$ **and** *sel*: select-highest $S = \text{Some } m$
shows $\forall m' \in S. \text{priority } m' \leq \text{priority } m$
proof –
from *highest-priority-exists*[*OF fin ne*] **have** *uniq*:
 $\exists!x. x \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } x)$.
from *theI'*[*OF uniq*] **have** *the-max*:
 $\forall m' \in S. \text{priority } m' \leq$
 $\text{priority } (\text{THE } x. x \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } x))$
by *auto*
from *sel ne* **have** $m = (\text{THE } x. x \in S \wedge (\forall m' \in S. \text{priority } m' \leq \text{priority } x))$
unfolding *select-highest-def* **by** *simp*
with *the-max* **show** *?thesis* **by** *simp*
qed
end

20 Deadlock-Free Locking with Timeout

A resource locking system where each lock has a bounded lifetime. After the timeout expires, the lock is considered released regardless of whether the holder explicitly released it. This models systems where Byzantine participants may hold locks indefinitely, but a timeout mechanism prevents permanent blocking.

The key property: no resource can be locked beyond its timeout, so no circular wait can persist indefinitely.

locale *deadlock-free-locking* =
fixes *timeout* :: nat
assumes *timeout-positive*: $\text{timeout} > 0$
begin

A lock is effective only if it has not expired. Once the current time exceeds the lock acquisition time plus timeout, the lock is no longer blocking.

definition *lock-effective* :: nat $\Rightarrow$ nat $\Rightarrow$ bool **where**
 $\text{lock-effective } \text{lock-time } \text{current-time} \iff \text{current-time} < \text{lock-time} + \text{timeout}$

Every lock eventually expires: for any lock acquired at time *t*, there exists a future time at which the lock is no longer effective.

theorem *lock-eventually-expires*:
 $\exists t'. t' \geq \text{current-time} \wedge \neg \text{lock-effective } \text{lock-time } t'$
proof
show $\text{lock-time} + \text{timeout} + \text{current-time} \geq \text{current-time} \wedge$
 $\neg \text{lock-effective } \text{lock-time } (\text{lock-time} + \text{timeout} + \text{current-time})$
unfolding *lock-effective-def* **by** *auto*
qed

The timeout is bounded: the lock expires no later than *lock-time* + *timeout* time units after acquisition.

lemma *lock-expiry-bound*:
assumes $\neg \text{lock-effective lock-time } t$
shows $t \geq \text{lock-time} + \text{timeout}$
using *assms* **unfolding** *lock-effective-def* **by** *auto*

At the exact expiry moment, the lock is no longer effective.

lemma *lock-expired-at-timeout*:
 $\neg \text{lock-effective lock-time } (\text{lock-time} + \text{timeout})$
unfolding *lock-effective-def* **by** *auto*

Before the timeout, the lock is effective.

lemma *lock-effective-before-timeout*:
assumes $\text{current-time} < \text{lock-time} + \text{timeout}$
shows $\text{lock-effective lock-time current-time}$
using *assms* **unfolding** *lock-effective-def* **by** *auto*

Deadlock freedom for single-resource locking: if each operation acquires at most one lock, and every lock has a bounded timeout, then no operation can be blocked indefinitely.

This follows directly from **lock_eventually_expires**: any lock that blocks an operation will expire within **timeout** time units. Since operations acquire single locks (no circular wait possible with single-resource locking), deadlock is structurally impossible.

theorem *deadlock-freedom*:
assumes $\text{lock-effective lock-time current-time}$
shows $\exists t'. t' \leq \text{lock-time} + \text{timeout} \wedge \neg \text{lock-effective lock-time } t'$
proof
show $\text{lock-time} + \text{timeout} \leq \text{lock-time} + \text{timeout} \wedge$
 $\neg \text{lock-effective lock-time } (\text{lock-time} + \text{timeout})$
using *lock-expired-at-timeout* **by** *auto*
qed
end

21 Fair Leader Starvation Freedom

A leader-based system where an honest leader always processes at least one pending request, and a fairness assumption guarantees that honest leaders appear periodically.

The fairness assumption is a sufficient condition that abstracts over the probabilistic guarantees of VRF-based leader election. Under $f < n / (3::'a)$ Byzantine faults, the probability that a Byzantine leader is elected k consecutive times is $(f / n)^k$, which decreases exponentially. The fairness bound k captures this as a deterministic assumption.

The key theorem: under the fairness assumption, any pending request is processed within a bounded number of epochs.

```

locale fair-leader-system =
  fixes leader-at :: nat  $\Rightarrow$  'n
    and is-honest :: 'n  $\Rightarrow$  bool
    and pending :: nat  $\Rightarrow$  nat
    and fairness-bound :: nat
  assumes fairness-bound-positive: fairness-bound > 0
  and fair-leader:
     $\forall$  epoch.  $\exists$  e. epoch  $\leq$  e  $\wedge$  e < epoch + fairness-bound  $\wedge$  is-honest (leader-at
e)
  and honest-progress:
     $\llbracket$  is-honest (leader-at e); pending e > 0  $\rrbracket \implies$  pending (Suc e) < pending e
  and non-honest-bounded:
    pending (Suc e)  $\leq$  pending e
begin

```

Pending count is monotonically non-increasing.

```

lemma pending-monotone:
  assumes e1  $\leq$  e2
  shows pending e2  $\leq$  pending e1
  using assms
proof (induction rule: inc-induct)
  case base
  then show ?case by simp
next
  case (step e)
  have pending e2  $\leq$  pending (Suc e) by (rule step.IH)
  also have pending (Suc e)  $\leq$  pending e using non-honest-bounded by auto
  finally show ?case .
qed

```

If there are pending requests, within *fairness-bound* epochs at least one will be processed (the pending count strictly decreases).

```

theorem starvation-bound:
  assumes pos: pending epoch > 0
  shows  $\exists$  e. epoch  $\leq$  e  $\wedge$  e < epoch + fairness-bound  $\wedge$  pending (Suc e) < pending
e
proof –
  from fair-leader obtain h where
    h-range: epoch  $\leq$  h  $\wedge$  h < epoch + fairness-bound and
    h-honest: is-honest (leader-at h)
  by auto
  show ?thesis
  proof (cases pending h > 0)
  case True
    with h-honest have pending (Suc h) < pending h using honest-progress by
auto
    with h-range show ?thesis by auto
  next
  case False

```

```

then have ph0: pending h = 0 by simp
— pending dropped from >0 (at epoch) to 0 (at h): find a strict decrease step
have key:  $\forall n. 0 < n \longrightarrow \text{pending}(\text{epoch} + n) = 0 \longrightarrow$ 
   $(\exists e. \text{epoch} \leq e \wedge e < \text{epoch} + n \wedge \text{pending}(\text{Suc } e) < \text{pending } e)$ 
proof (rule allI)
  fix n
  show  $0 < n \longrightarrow \text{pending}(\text{epoch} + n) = 0 \longrightarrow$ 
     $(\exists e. \text{epoch} \leq e \wedge e < \text{epoch} + n \wedge \text{pending}(\text{Suc } e) < \text{pending } e)$ 
  proof (induction n)
    case 0 show ?case by simp
  next
    case (Suc k)
    show ?case
    proof (intro impI)
      assume hpk: pending (epoch + Suc k) = 0
      show  $\exists e. \text{epoch} \leq e \wedge e < \text{epoch} + \text{Suc } k \wedge \text{pending}(\text{Suc } e) < \text{pending } e$ 
      proof (cases k = 0)
        case True
        from hpk True have pending (Suc epoch) < pending epoch using pos by
simp
          thus ?thesis by (intro exI[of - epoch]) auto
        next
          case False
          then have kpos:  $0 < k$  by simp
          show ?thesis
          proof (cases pending (epoch + k) = 0)
            case True
            then have pk0: pending (epoch + k) = 0 .
            from Suc.IH[THEN mp, OF kpos, THEN mp, OF pk0]
            obtain e' where  $\text{epoch} \leq e' \wedge e' < \text{epoch} + k \wedge \text{pending}(\text{Suc } e') < \text{pending}$ 
e'
              by auto
            thus ?thesis by auto
          next
            case False
            from False hpk have pending (epoch + Suc k) < pending (epoch + k)
by simp
              thus ?thesis by (intro exI[of - epoch + k]) auto
            qed
          qed
        qed
      qed
    qed
  have dpos:  $0 < h - \text{epoch}$ 
  proof —
    have  $h \neq \text{epoch}$  using ph0 pos by auto
    with h-range(1) show ?thesis by arith
  qed
  have pd: pending (epoch + (h - epoch)) = 0

```

```

proof –
  have  $epoch + (h - epoch) = h$  using  $h\text{-range}(1)$  by  $arith$ 
  with  $ph0$  show  $?thesis$  by  $simp$ 
qed
from  $key[rule\text{-}format, OF\ dpos, OF\ pd]$ 
obtain  $e'$  where  $he': epoch \leq e' \ e' < epoch + (h - epoch)$   $pending\ (Suc\ e') <$ 
 $pending\ e'$ 
  by  $auto$ 
  have  $epoch + (h - epoch) \leq h$  using  $h\text{-range}(1)$  by  $arith$ 
  with  $he' h\text{-range}(2)$  show  $?thesis$  by  $auto$ 
qed
qed

```

Corollary: all pending requests are eventually processed. By induction on the pending count (which is a natural number and thus a well-founded decreasing measure), repeated application of `starvation_bound` drives the count to zero.

theorem *eventual-completion*:

shows $\exists e\text{-final}. pending\ e\text{-final} = 0$

proof –

define P **where**

$P\ n \equiv \forall epoch. pending\ epoch = n \longrightarrow (\exists e\text{-final}. pending\ e\text{-final} = 0)$

for $n :: nat$

have $all\text{-}P: \bigwedge n. P\ n$

proof –

fix n **show** $P\ n$

unfolding $P\text{-def}$

proof (*induction* n *rule*: *less-induct*)

case (*less* n)

show $\forall epoch. pending\ epoch = n \longrightarrow (\exists e\text{-final}. pending\ e\text{-final} = 0)$

proof (*intro* *allI* *impI*)

fix $epoch$ **assume** $eq: pending\ epoch = n$

show $\exists e\text{-final}. pending\ e\text{-final} = 0$

proof (*cases* $n = 0$)

case *True* **thus** $?thesis$ **using** eq **by** $auto$

next

case *False*

have $pos: pending\ epoch > 0$ **using** eq *False* **by** $auto$

from $starvation\text{-}bound[OF\ pos]$ **obtain** e **where**

$e\text{-ge}: epoch \leq e$ **and** $e\text{-dec}: pending\ (Suc\ e) < pending\ e$

by $auto$

have $lt: pending\ (Suc\ e) < n$

using $pending\text{-}monotone[OF\ e\text{-ge}]$ $eq\ e\text{-dec}$ **by** $simp$

from $less.IH[OF\ lt]$ **show** $?thesis$

unfolding $P\text{-def}$ **by** $blast$

qed

qed

qed

qed

```

    from all-P[of pending 0] show ?thesis
      unfolding P-def by blast
qed

end

end

```

```

theory DQuencer-Instance
  imports Priority-Resolution Regulatory-Instance HOL-Library.Product-Lexorder
begin

```

22 Authority Level and Action Severity

Regulatory authority hierarchy. International authorities (e.g., FATF) take precedence over national (e.g., SEC), which take precedence over regional authorities. This reflects the RCP framework’s jurisdictional priority model.

```

datatype authority-level = Regional | National | International

```

```

fun authority-rank :: authority-level  $\Rightarrow$  nat where
  authority-rank Regional = 1
| authority-rank National = 2
| authority-rank International = 3

```

```

lemma authority-rank-injective:
  authority-rank a1 = authority-rank a2  $\implies$  a1 = a2
  by (cases a1; cases a2; auto)

```

Action severity determines priority among actions from the same authority level. Stronger enforcement actions (CONFISCATE, SEIZE) take precedence over weaker ones (RESTRICT, UNFREEZE). This reflects the legal principle that more conservative (asset-protective) actions should prevail when concurrent conflicting orders exist.

```

fun action-severity :: reg-action  $\Rightarrow$  nat where
  action-severity UNRESTRICT = 1
| action-severity UNFREEZE = 2
| action-severity RELEASE = 3
| action-severity RESTRICT = 4
| action-severity FREEZE = 5
| action-severity SEIZE = 6
| action-severity CONFISCATE = 7

```

```

lemma action-severity-injective:
  action-severity a1 = action-severity a2  $\implies$  a1 = a2
  by (cases a1; cases a2; auto)

```

23 Priority Key

The priority key is a 4-tuple of natural numbers, using Isabelle's built-in lexicographic order on products. No manual `linorder` instance registration is needed.

Components (highest to lowest significance):

1. `authority_rank`: higher authority = higher priority
2. `inverted_timestamp`: earlier timestamp = higher priority (inverted)
3. `action_severity`: stronger action = higher priority
4. `node_id`: deterministic tiebreaker (lower node ID = higher priority, so we invert to make larger = higher in the `nat` order)

For timestamp and `node_id` inversion, we use *max-val - actual-val* to convert "smaller is better" to "larger is better" for compatibility with the `nat` product order where larger = higher.

type-synonym *priority-key* = *nat* × *nat* × *nat* × *nat*

24 Extended Message Type

Extends `oss_message` from `Regulatory_Instance` with authority level and source node ID. Uses Isabelle record inheritance so that existing functions on `oss_message` remain applicable.

record *dq-message* = *oss-message* +
 dqm-authority-level :: *authority-level*
 dqm-source-node :: *nat*

25 Priority Construction

Construct a priority key from message fields. Uses two upper bounds for inversion: `max_time` for timestamps and `max_node` for node IDs.

definition *make-priority-key* ::
 nat ⇒ *nat* ⇒ *dq-message* ⇒ *priority-key*
where
 make-priority-key max-time max-node msg =
 (*authority-rank* (*dqm-authority-level* *msg*),
 max-time - *msg-timestamp* *msg*,
 action-severity (*msg-action* *msg*),
 max-node - *dqm-source-node* *msg*)

26 Node and System Model

datatype *node-behavior* = *Honest* | *Byzantine*

record *node-info* =
 ni-id :: nat
 ni-behavior :: *node-behavior*

definition *honest-nodes* :: *node-info set* $\Rightarrow$ *node-info set* **where**
 honest-nodes ns = {*n* $\in$ *ns*. *ni-behavior n* = *Honest*}

definition *byzantine-nodes* :: *node-info set* $\Rightarrow$ *node-info set* **where**
 byzantine-nodes ns = {*n* $\in$ *ns*. *ni-behavior n* = *Byzantine*}

lemma *honest-byzantine-partition*:
 honest-nodes ns $\cup$ *byzantine-nodes ns* = *ns*
proof (*rule set-eqI*)
 fix *x*
 show *x* $\in$ *honest-nodes ns* $\cup$ *byzantine-nodes ns* $\longleftrightarrow$ *x* $\in$ *ns*
 unfolding *honest-nodes-def byzantine-nodes-def*
 by (*cases ni-behavior x; auto*)
qed

lemma *honest-byzantine-disjoint*:
 honest-nodes ns $\cap$ *byzantine-nodes ns* = {}
 unfolding *honest-nodes-def byzantine-nodes-def* **by** *auto*

27 D-quencer System Locale

The D-quencer system locale encapsulates the BFT assumption $((3::'a) * f + 1 \leq n)$ and system parameters.

locale *dquencer-system* =
 fixes *nodes* :: *node-info set*
 and *f-max* :: nat
 and *lock-timeout* :: nat
 and *fairness-bound* :: nat
 and *max-time* :: nat
 and *max-node* :: nat
 assumes *finite-nodes*: *finite nodes*
 and *bft-threshold*: *card nodes* $\geq 3 * f\text{-max} + 1$
 and *byzantine-bound*: *card (byzantine-nodes nodes)* $\leq f\text{-max}$
 and *nonempty-nodes*: *nodes* $\neq \{\}$
 and *timeout-positive*: *lock-timeout* > 0
 and *fairness-positive*: *fairness-bound* > 0
begin

lemma *honest-majority*:
 card (honest-nodes nodes) $> 2 * f\text{-max}$

```

proof –
  have card nodes = card (honest-nodes nodes) + card (byzantine-nodes nodes)
  using honest-byzantine-partition honest-byzantine-disjoint finite-nodes
  by (metis card-Un-disjoint finite-Un)
  with bft-threshold byzantine-bound show ?thesis by linarith
qed

```

```

lemma honest-nonempty:
  honest-nodes nodes ≠ {}
proof –
  have fin: finite (honest-nodes nodes)
  unfolding honest-nodes-def using finite-nodes by simp
  have pos: 0 < card (honest-nodes nodes)
  using honest-majority by linarith
  with fin pos show ?thesis by (auto simp: card-eq-0-iff)
qed

```

end

28 Priority System Instantiation

We instantiate the `priority_system` locale from `Priority_Resolution.thy` with D-quencer message priorities.

The priority function maps messages to `priority_key` (which is `nat \<times> nat \<times> nat \<times> nat` with the built-in lexicographic order).

Injectivity follows from: distinct messages have either different authority levels, different timestamps, different actions, or different source nodes. The source node ID serves as the final tiebreaker ensuring no two distinct messages share the same key.

For a concrete message set with the injectivity precondition, the `priority_system` locale provides deterministic selection.

```

lemma priority-key-injectivity:
  assumes make-priority-key mt mn m1 = make-priority-key mt mn m2
  and msg-timestamp m1 ≤ mt and msg-timestamp m2 ≤ mt
  and dqm-source-node m1 ≤ mn and dqm-source-node m2 ≤ mn
  shows dqm-authority-level m1 = dqm-authority-level m2
   $\wedge$  msg-timestamp m1 = msg-timestamp m2
   $\wedge$  msg-action m1 = msg-action m2
   $\wedge$  dqm-source-node m1 = dqm-source-node m2
proof –
  from assms(1) have
    eq1: authority-rank (dqm-authority-level m1) = authority-rank (dqm-authority-level m2) and
    eq2: mt - msg-timestamp m1 = mt - msg-timestamp m2 and
    eq3: action-severity (msg-action m1) = action-severity (msg-action m2) and
    eq4: mn - dqm-source-node m1 = mn - dqm-source-node m2

```

```

  unfolding make-priority-key-def by auto
from eq1 have al: dqm-authority-level m1 = dqm-authority-level m2
  using authority-rank-injective by auto
from eq2 assms(2,3) have ts: msg-timestamp m1 = msg-timestamp m2
  by linarith
from eq3 have act: msg-action m1 = msg-action m2
  using action-severity-injective by auto
from eq4 assms(4,5) have nd: dqm-source-node m1 = dqm-source-node m2
  by linarith
from al ts act nd show ?thesis by auto
qed

```

Auxiliary corollary: under the well-formedness bounds, equal priority keys imply equal messages on every priority-relevant field.

The generic **priority_system** locale assumes priority injectivity unconditionally on its carrier type. We instantiate it with the identity function on **priority_key** as the carrier; injectivity holds by reflexivity on the carrier. The recovery of the unique D-quencer message corresponding to a chosen priority key is provided as a corollary inside the **dquencer_priority_concrete** locale, using the well-formedness bounds and the additional priority-key distinctness assumption.

```

interpretation dq-priority:
  priority-system id :: priority-key  $\Rightarrow$  priority-key
  by unfold-locales auto

```

The **dquencer_priority_context** locale extends **dquencer_system** with a fixed message set whose elements satisfy the well-formedness bounds. Together with the bound-preservation, this set induces the priority key set **msg_priority_keys** on which priority-based selection operates.

```

locale dquencer-priority-context = dquencer-system +
  fixes msg-set :: dq-message set
  assumes msg-set-nonempty: msg-set  $\neq$  {}
  and msg-set-well-formed:
     $\bigwedge m. m \in \text{msg-set} \implies \text{msg-timestamp } m \leq \text{max-time}$ 
     $\wedge \text{dqm-source-node } m \leq \text{max-node}$ 

```

begin

```

definition msg-priority-keys :: priority-key set where
  msg-priority-keys = make-priority-key max-time max-node ' msg-set

```

```

lemma msg-priority-keys-nonempty: msg-priority-keys  $\neq$  {}
  using msg-set-nonempty unfolding msg-priority-keys-def by simp

```

```

lemma msg-priority-keys-finite:
  finite msg-set  $\implies$  finite msg-priority-keys
  unfolding msg-priority-keys-def by simp

```

end

The `dquencer_priority_concrete` locale strengthens the context with the priority-key distinctness assumption on `msg_set`. This is the natural BFT-consensus precondition: distinct authority / timestamp / severity / source-node tuples ensure determinism. Under this assumption the priority key uniquely identifies a message in `msg_set`, so selecting the highest priority key in `msg_priority_keys` is equivalent to selecting the highest-priority message in `msg_set`.

```

locale dquencer-priority-concrete = dquencer-priority-context +
  assumes msg-set-priority-distinct:
     $\bigwedge m1\ m2. m1 \in \text{msg\_set} \implies m2 \in \text{msg\_set}$ 
     $\implies \text{make-priority-key max-time max-node } m1$ 
     $= \text{make-priority-key max-time max-node } m2$ 
     $\implies m1 = m2$ 

```

begin

Recovery: given a priority key from `msg_priority_keys`, return the unique `msg_set` message with that key.

definition *recover-msg* :: *priority-key* $\Rightarrow$ *dq-message* **where**
recover-msg $k = (\text{THE } m. m \in \text{msg_set} \wedge \text{make-priority-key max-time max-node } m = k)$

lemma *recover-msg-unique-existence*:

```

assumes  $k \in \text{msg-priority-keys}$ 
shows  $\exists! m. m \in \text{msg\_set} \wedge \text{make-priority-key max-time max-node } m = k$ 

```

proof –

```

from assms obtain  $m$ 
  where  $m\text{-in}: m \in \text{msg\_set}$ 
  and  $m\text{-key}: \text{make-priority-key max-time max-node } m = k$ 
  unfolding msg-priority-keys-def by auto
show ?thesis
proof (rule ex1I[of - m])
  show  $m \in \text{msg\_set} \wedge \text{make-priority-key max-time max-node } m = k$ 
    using  $m\text{-in } m\text{-key}$  by auto
next
  fix  $m'$ 
  assume  $m' \in \text{msg\_set} \wedge \text{make-priority-key max-time max-node } m' = k$ 
  then have  $m'\text{-in}: m' \in \text{msg\_set}$ 
    and  $m'\text{-key}: \text{make-priority-key max-time max-node } m' = k$ 
    by auto
  from  $m\text{-key } m'\text{-key}$ 
  have  $\text{make-priority-key max-time max-node } m'$ 
     $= \text{make-priority-key max-time max-node } m$ 
    by simp
  then show  $m' = m$ 
    using msg-set-priority-distinct[OF m'-in m-in] by simp
qed
qed

```

lemma *recover-msg-correct*:
assumes $k \in \text{msg-priority-keys}$
shows $\text{recover-msg } k \in \text{msg-set}$
 $\wedge \text{make-priority-key max-time max-node } (\text{recover-msg } k) = k$
proof –
from *recover-msg-unique-existence*[*OF assms*]
have $\text{ex1}: \exists!m. m \in \text{msg-set} \wedge \text{make-priority-key max-time max-node } m = k$.
show *?thesis*
using *theI*'[*OF ex1*] **unfolding** *recover-msg-def*.
qed

Concrete consequence: deterministic selection of the highest-priority key from a non-empty finite `msg_priority_keys` set, and recovery of the unique well-formed D-quencer message that produced it.

corollary *dq-select-highest-deterministic*:
assumes *finite msg-set*
shows $\exists!k. \text{dq-priority.select-highest msg-priority-keys} = \text{Some } k$
using *dq-priority.select-highest-deterministic*
 $[\text{OF msg-priority-keys-finite}[\text{OF assms}] \text{ msg-priority-keys-nonempty}]$.

corollary *dq-select-highest-in-set*:
assumes *finite msg-set*
and $\text{dq-priority.select-highest msg-priority-keys} = \text{Some } k$
shows $k \in \text{msg-priority-keys}$
using *dq-priority.select-highest-in-set*
 $[\text{OF msg-priority-keys-finite}[\text{OF assms}(1)] \text{ assms}(2)]$.

corollary *dq-select-highest-message*:
assumes *finite msg-set*
and $\text{dq-priority.select-highest msg-priority-keys} = \text{Some } k$
shows $\text{recover-msg } k \in \text{msg-set}$
 $\wedge \text{make-priority-key max-time max-node } (\text{recover-msg } k) = k$
using *recover-msg-correct*[*OF dq-select-highest-in-set*[*OF assms*]].

end

29 BFT Consensus Abstraction

We abstract the BFT consensus mechanism. Rather than formalizing BLS signatures and vote counting, we model the consensus as: given a set of messages and honest majority, the consensus output is the highest-priority valid message.

This is justified because:

1. Honest nodes follow the priority protocol
2. With $> 2/3$ honest nodes, the BFT consensus output reflects the honest majority's agreement

3. The honest majority will agree on the highest-priority message (since priority is a total order and deterministic)

definition *valid-dq-message* :: *global-state* $\Rightarrow$ *dq-message* $\Rightarrow$ *bool* **where**
valid-dq-message *gs* *msg* =
 (*asset-exists* *gs* (*msg-source* *msg*) (*msg-asset-id* *msg*) $\wedge$
 ($\exists s. \text{get-reg-state } gs \text{ (msg-source } msg) \text{ (msg-asset-id } msg) = \text{Some } s \wedge$
reg-transition *s* (*msg-action* *msg*) $\neq \text{None}$))

A scalar encoding of the priority key for use with **sort_key**. Maps each message to a single natural number that preserves the lexicographic order on priority key components, given the bounds *max-t* and *max-n*.

definition *priority-scalar* ::
nat $\Rightarrow$ *nat* $\Rightarrow$ *dq-message* $\Rightarrow$ *nat*
where
priority-scalar *max-t* *max-n* *m* =
authority-rank (*dqm-authority-level* *m*) * (*Suc* *max-t*) * 8 * (*Suc* *max-n*) +
 (*max-t* - *msg-timestamp* *m*) * 8 * (*Suc* *max-n*) +
action-severity (*msg-action* *m*) * (*Suc* *max-n*) +
 (*max-n* - *dqm-source-node* *m*)

definition *bft-select* ::
dq-message list $\Rightarrow$ *global-state* $\Rightarrow$ *nat* $\Rightarrow$ *nat* $\Rightarrow$ *dq-message option*
where
bft-select *msgs* *gs* *max-t* *max-n* =
 (let *valid-msgs* = *filter* (*valid-dq-message* *gs*) *msgs*;
 ranked = *sort-key* (*priority-scalar* *max-t* *max-n*) *valid-msgs*
 in if *ranked* = [] then *None* else *Some* (*last* *ranked*))

30 Deadlock-Free Locking Instantiation

We instantiate the **deadlock_free_locking** locale with the D-quencer's lock timeout. This provides:

1. Every lock expires within **lock_timeout** time units
2. Byzantine nodes that hold locks indefinitely are handled by forced timeout release
3. Deadlock freedom: no asset can be permanently blocked

context *dquencer-system*
begin

The D-quencer system's lock timeout satisfies the **deadlock_free_locking** locale.

interpretation *dq-locking*: *deadlock-free-locking lock-timeout*

by *unfold-locales (rule timeout-positive)*

Concrete deadlock freedom for the D-quencer: any locked asset will be unlocked within `lock_timeout` time.

corollary *dq-deadlock-freedom:*

assumes *dq-locking.lock-effective lock-time current-time*

shows $\exists t'. t' \leq \text{lock-time} + \text{lock-timeout} \wedge$
 $\neg \text{dq-locking.lock-effective lock-time } t'$

using *dq-locking.deadlock-freedom[OF assms]* .

Byzantine lock resistance: even if a Byzantine node acquires a lock and never releases it, the lock expires by timeout.

corollary *byzantine-lock-expires:*

$\exists t'. \neg \text{dq-locking.lock-effective lock-time } t'$

using *dq-locking.lock-eventually-expires* **by** *auto*

end

31 Fair Leader Starvation Freedom Instantiation

We instantiate the `fair_leader_system` locale with the D-quencer's leader election and pending request processing.

The fairness assumption abstracts VRF randomness: within any `fairness_bound` consecutive epochs, at least one epoch has an honest leader. Under $f < n / (3::'a)$, the probability of *fairness-bound* consecutive Byzantine leaders is $(f / n)^{\text{fairness-bound}} < (1 / (3::'a))^{\text{fairness-bound}}$.

For concrete instantiation, we need a leader schedule and pending count function that satisfy the locale assumptions. These are provided as parameters in the instantiation context.

locale *dquencer-liveness = dquencer-system +*

fixes *leader-schedule :: nat $\Rightarrow$ node-info*

and *pending-count :: nat $\Rightarrow$ nat*

assumes *fair-leader:*

$\forall \text{epoch}. \exists e. \text{epoch} \leq e \wedge e < \text{epoch} + \text{fairness-bound} \wedge$
 $\text{ni-behavior (leader-schedule } e) = \text{Honest}$

and *honest-processes:*

$\llbracket \text{ni-behavior (leader-schedule } e) = \text{Honest}; \text{pending-count } e > 0 \rrbracket$
 $\implies \text{pending-count (Suc } e) < \text{pending-count } e$

and *pending-non-increasing:*

$\text{pending-count (Suc } e) \leq \text{pending-count } e$

begin

interpretation *dq-fair: fair-leader-system*

leader-schedule $\lambda n. \text{ni-behavior } n = \text{Honest}$ pending-count fairness-bound

proof *unfold-locales*

show $0 < \text{fairness-bound}$ **using** *fairness-positive* .

```

next
  show  $\forall epoch. \exists e. epoch \leq e \wedge e < epoch + \text{fairness-bound} \wedge$ 
     $ni\text{-behavior}(\text{leader-schedule } e) = \text{Honest}$ 
    using fair-leader .
next
  fix  $e$ 
    assume  $ni\text{-behavior}(\text{leader-schedule } e) = \text{Honest}$   $0 < \text{pending-count } e$ 
    then show  $\text{pending-count}(\text{Suc } e) < \text{pending-count } e$ 
    using honest-processes by auto
next
  fix  $e$ 
    show  $\text{pending-count}(\text{Suc } e) \leq \text{pending-count } e$ 
    using pending-non-increasing .
qed

```

Concrete starvation freedom for the D-quencer: if there are pending regulatory requests, at least one will be processed within `fairness_bound` epochs.

```

corollary dq-starvation-bound:
  assumes  $\text{pending-count } epoch > 0$ 
  shows  $\exists e. epoch \leq e \wedge e < epoch + \text{fairness-bound} \wedge$ 
     $\text{pending-count}(\text{Suc } e) < \text{pending-count } e$ 
  using dq-fair.starvation-bound[OF assms] .

```

end

32 Combined Safety and Liveness

This section connects Property 1 (cross-domain state preservation, safety) with Property 2 (determinism + liveness).

Property 1 guarantees: if sync is executed correctly, the resulting state preserves cross-domain consistency.

Property 2 guarantees: under Byzantine faults, D-quencer deterministically selects actions, locks are bounded by timeout, and all pending requests are eventually processed.

Combined: in a decentralized Byzantine environment, cross-domain regulatory state is synchronized deterministically, without deadlock, and without starvation.

The combined theorem ties `valid_state_preservation` from `Regulatory_Instance.thy` with the liveness guarantees from this theory.

```

theorem combined-safety-liveness:
  assumes valid: valid-state gs
  and exists: asset-exists gs source aid
  and current: get-reg-state gs source aid = Some s
  and trans: reg-transition s action = Some s'
  and not-locked:  $\neg$  is-locked gs aid

```



```

    and fin: finite (connected-chains gs aid)
  shows  $\exists gs'. \text{sync source action aid gs} = \text{Some gs}' \wedge \text{valid-state gs}'$ 
proof -
  from not-locked obtain gs-locked where
    lock: acquire-lock gs aid = Some gs-locked
  using lock-acquire-success by auto
  from valid current trans lock have
     $\exists gs'. \text{sync source action aid gs} = \text{Some gs}'$ 
  unfolding sync-def
  using exists current trans lock
  by (auto simp: get-reg-state-def get-asset-state-def
      split: option.splits
      intro!: exI)
  then obtain gs' where synced: sync source action aid gs = Some gs'
  by auto
  have valid-state gs'
  using valid-state-preservation[OF valid exists current trans synced fin] .
  with synced show ?thesis by auto
qed

```

Summary of what Properties 1 + 2 together establish:

1. **Safety** (Property 1): After synchronization, all connected chains agree on the regulatory state. The global validity invariant is preserved.
2. **Determinism** (Property 2): Conflicting regulatory actions are resolved by a total order on priority keys. The BFT consensus output is unique. The `priority_system` locale is instantiated on the `priority_key` type as carrier (with the corresponding messages recovered inside `dquencer_priority_concrete` via `recover_msg`), yielding deterministic selection from any finite non-empty set of valid candidate messages.
3. **Deadlock freedom** (Property 2): No asset can be permanently locked, even if Byzantine nodes refuse to release locks. Timeout-based forced release bounds the lock duration.
4. **Starvation freedom** (Property 2): Under the fair leader assumption, every pending regulatory request is processed within a bounded number of epochs.

Open work for subsequent entries: - Compositional assurance across heterogeneous verification regimes (Canton + OSS + EVM). - Refinement: formal correspondence between these models and the Rust / Solidity implementation.

end

33 Acknowledgments

The regulatory state model (five states, seven actions, and the transition exclusions) is motivated by the Regulatory Compliance Protocol (RCP), a framework developed by the Oraclizer research team that systematically classifies requirements from 15 global financial regulatory authorities for tokenized capital markets. RCP is currently proposed as an Informational Ethereum Improvement Proposal (EIP) in Draft status.